

```

        self.kernel =
            self.add_weight( name="kernel",
shape=[batch_input_shape[-1], self.units], initializer="glorot_normal")
        self.bias = self.add_weight(
            name="bias", shape=[self.units], initializer="zeros")

    def call(self, X):
        return self.activation(X @ self.kernel + self.bias)

    def get_config(self):
        base_config = super().get_config() return
        {**base_config, "units": self.units, "activation":

tf.keras.activations.serialize(self.activation)}

```

Hãy cùng phân tích đoạn mã này:

- Hàm khởi tạo nhận tất cả các siêu tham số làm đối số (trong ví dụ này, units và activation), và quan trọng là nó cũng nhận một đối số `**kwargs`. Nó gọi hàm khởi tạo của lớp cha, truyền cho nó các đối số kwargs: điều này xử lý các đối số tiêu chuẩn như input_shape, trainable và name. Sau đó, nó lưu các siêu tham số dưới dạng thuộc tính, chuyển đổi đối số activation thành hàm kích hoạt thích hợp bằng cách sử dụng hàm `tf.keras.activations.get()` (nó chấp nhận các hàm, các chuỗi tiêu chuẩn như "relu" hoặc "swish", hoặc đơn giản là None).
- Vai trò của phương thức `build()` là tạo ra các biến của lớp bằng cách gọi phương thức `add_weight()` cho mỗi trọng số. Phương thức `build()` được gọi lần đầu tiên khi lớp được sử dụng. Tại thời điểm đó, Keras sẽ biết hình dạng của đầu vào của lớp này và sẽ truyền nó cho phương thức `build()`¹⁰, điều này thường cần thiết để tạo ra một số trọng số. Ví dụ, chúng ta cần biết số lượng neuron trong lớp trước đó để tạo ma trận trọng số kết nối (tức là "kernel"): điều này tương ứng với kích thước của chiều cuối cùng của đầu vào. Ở cuối phương thức `build()` (và chỉ ở cuối), bạn phải gọi phương thức `build()` của lớp cha: điều này cho Keras biết rằng lớp đã được xây dựng (nó chỉ đặt `self.built = True`).

- Phương thức `call()` thực hiện các thao tác mong muốn. Trong trường hợp này, chúng ta tính phép nhân ma trận giữa đầu vào `X` và kernel của lớp, cộng thêm vectơ bias, và áp dụng hàm kích hoạt cho kết quả, và điều này cho chúng ta đầu ra của lớp.
- Phương thức `get_config()` tương tự như trong các lớp tùy chỉnh trước đó. Lưu ý rằng chúng ta lưu cấu hình đầy đủ của hàm kích hoạt bằng cách gọi `tf.keras.activations.serialize()`.

Giờ đây bạn có thể sử dụng lớp `MyDense` giống như bất kỳ lớp nào khác!

GHI CHÚ

Keras tự động suy ra hình dạng đầu ra, ngoại trừ trường hợp lớp là động (như chúng ta sẽ thấy ngay sau đây). Trong trường hợp (hiếm gặp) này, bạn cần triển khai phương thức `compute_output_shape()`, phương thức này phải trả về một đối tượng `TensorShape`.

Để tạo một lớp có nhiều đầu vào (ví dụ: `Concatenate`), đối số của phương thức `call()` phải là một tuple chứa tất cả các đầu vào. Để tạo một lớp có nhiều đầu ra, phương thức `call()` phải trả về danh sách các đầu ra. Ví dụ, lớp đơn giản sau đây nhận hai đầu vào và trả về ba đầu ra:

```
class MyMultiLayer(tf.keras.layers.Layer): def
    call(self, X): X1,
        X2 = X
        return X1 + X2, X1 * X2, X1 / X2
```

Lớp này giờ đây có thể được sử dụng như bất kỳ lớp nào khác, nhưng tất nhiên chỉ sử dụng API Chức năng và Kế thừa lớp, chứ không phải API Tuần tự (chỉ chấp nhận các lớp có một đầu vào và một đầu ra).

Nếu lớp của bạn cần có hành vi khác nhau trong quá trình huấn luyện và trong quá trình kiểm thử (ví dụ: nếu nó sử dụng các lớp `Dropout` hoặc `BatchNormalization`), thì bạn phải thêm một đối số huấn luyện vào phương thức `call()` và sử dụng đối số này để quyết định phải làm gì. Ví dụ, hãy tạo một lớp thêm...

Nhiều Gaussian được sử dụng trong quá trình huấn luyện (để điều chỉnh) nhưng không có tác dụng gì trong quá trình kiểm thử (Keras có một lớp thực hiện điều tương tự, `tf.keras.layers.GaussianNoise`):

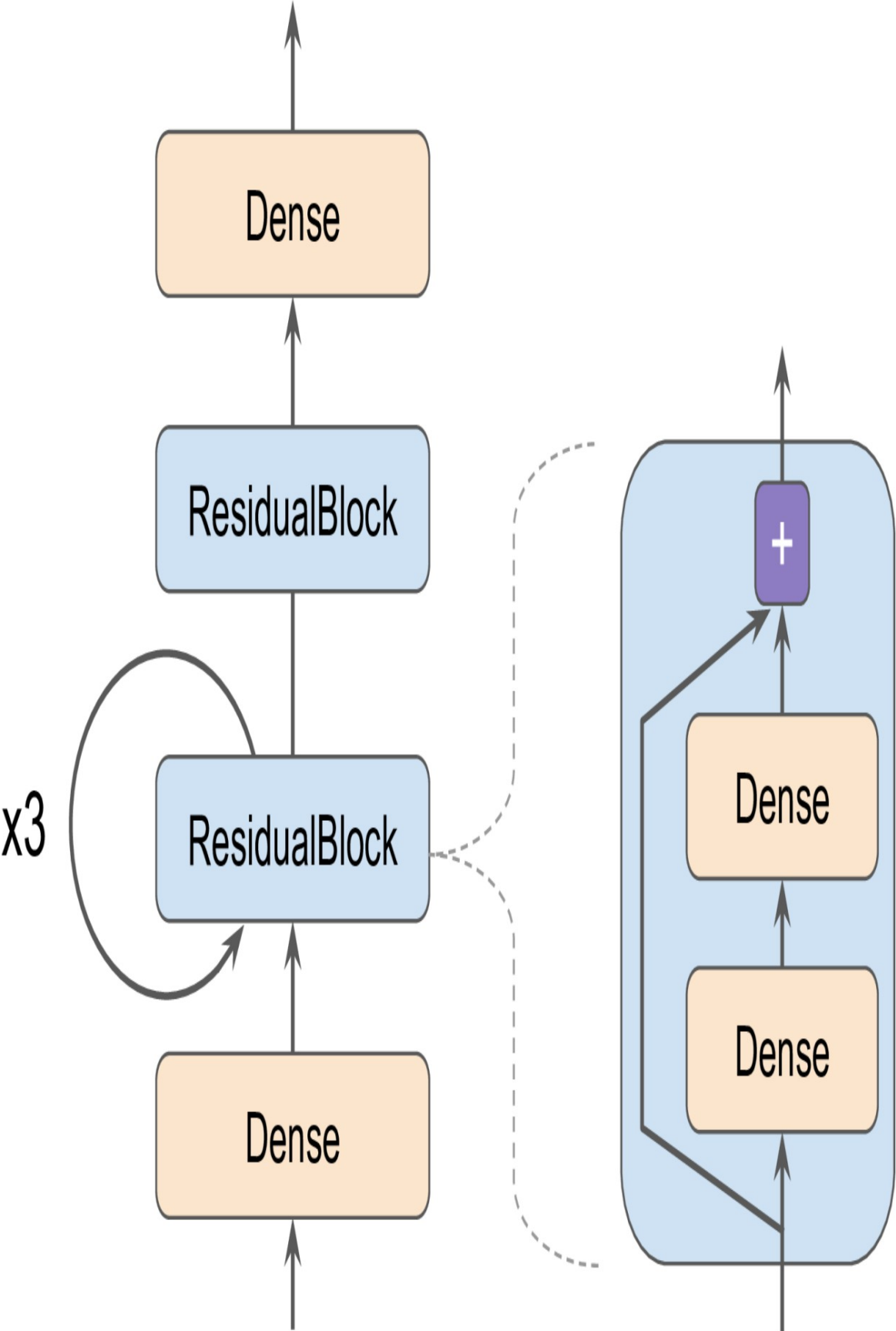
```
class MyGaussianNoise(tf.keras.layers.Layer):
    def __init__(self, stddev, **kwargs):
        super().__init__(**kwargs)
        self.stddev = stddev

    def call(self, X, training=False):
        if training:
            noise = tf.random.normal(tf.shape(X),
                                     stddev=self.stddev)
            # Trả về X + nhiễu nếu không
            # thì:
            # trả về X
```

Như vậy, giờ bạn có thể tạo bất kỳ lớp tùy chỉnh nào bạn cần! Bây giờ hãy cùng tạo các mô hình tùy chỉnh.

Mô hình tùy chỉnh

Chúng ta đã xem xét việc tạo các lớp mô hình tùy chỉnh trong [Chương 10](#), khi chúng ta đã thảo luận về API kế thừa lớp ¹¹`tf.keras.Model`. Nó khá đơn giản: kế thừa lớp `tf.keras.Model`, tạo các lớp và biến trong hàm tạo, và triển khai phương thức `call()` để thực hiện bất cứ điều gì bạn muốn mô hình thực hiện. Ví dụ, giả sử bạn muốn xây dựng mô hình được thể hiện trong [Hình 12-3](#).



Hình 12-3. Ví dụ về mô hình tùy chỉnh: một mô hình bất kỳ với lớp ResidualBlock tùy chỉnh chứa kết nối bỏ qua.

Các tín hiệu đầu vào đi qua một lớp dày đặc đầu tiên, sau đó qua một khối dư gồm hai lớp dày đặc và một phép cộng (như chúng ta sẽ thấy trong [Chương 14](#), một khối dư cộng các tín hiệu đầu vào với các tín hiệu đầu ra của nó), sau đó đi qua cùng khối dư này thêm ba lần nữa, rồi qua một khối dư thứ hai, và kết quả cuối cùng đi qua một lớp đầu ra dày đặc. Đừng lo lắng nếu mô hình này không dễ hiểu; nó chỉ là một ví dụ để minh họa rằng bạn có thể dễ dàng xây dựng bất kỳ loại mô hình nào bạn muốn, thậm chí cả mô hình có chứa vòng lặp và kết nối bỏ qua. Để triển khai mô hình này, tốt nhất là trước tiên nên tạo một lớp ResidualBlock, vì chúng ta sẽ tạo một vài khối giống hệt nhau (và chúng ta có thể muốn sử dụng lại nó trong một mô hình khác):

```
class ResidualBlock(tf.keras.layers.Layer):
    def __init__(self, n_layers, n_neurons, **kwargs):
        super().__init__(**kwargs)
        self.hidden = [tf.keras.layers.Dense(n_neurons,
activation="relu",

kernel_initializer="he_normal") cho
        _ trong phạm vi (n_lớp)]

    def call(self, inputs): Z =
        inputs for
        layer in self.hidden: Z =
            layer(Z)
        return inputs + Z
```

Lớp này hơi đặc biệt vì nó chứa các lớp khác. Keras xử lý điều này một cách minh bạch: nó tự động phát hiện thuộc tính ẩn chứa các đối tượng có thể theo dõi (các lớp trong trường hợp này), vì vậy các biến của chúng được tự động thêm vào danh sách biến của lớp này. Phần còn lại của lớp này khá dễ hiểu. Tiếp theo, hãy sử dụng API kế thừa lớp con để định nghĩa chính mô hình:

```
class ResidualRegressor(tf.keras.Model):
    def __init__(self, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = tf.keras.layers.Dense(30,
```

```

kích hoạt="relu",

kernel_initializer="he_normal")
self.block1 = ResidualBlock(2, 30) self.block2
= ResidualBlock(2, 30) self.out =
tf.keras.layers.Dense(output_dim)

def call(self, inputs): Z =
self.hidden1(inputs) for in
range(1 + 3): Z = self.block1(Z)

Z = self.block2(Z)
return self.out(Z)

```

Chúng ta tạo các lớp trong hàm tạo và sử dụng chúng trong phương thức `call()`. Sau đó, mô hình này có thể được sử dụng như bất kỳ mô hình nào khác (biên dịch, huấn luyện, đánh giá và sử dụng để đưa ra dự đoán). Nếu bạn cũng muốn có thể lưu mô hình bằng phương thức `save()` và tải nó bằng hàm `tf.keras.models.load_model()`, bạn phải triển khai phương thức `get_config()` (như chúng ta đã làm trước đó) trong cả lớp `ResidualBlock` và lớp `ResidualRegressor`. Ngoài ra, bạn cũng có thể lưu và tải các trọng số bằng cách sử dụng các phương thức `save_weights()` và `load_weights()`.

Lớp `Model` là lớp con của lớp `Layer`, vì vậy các model có thể được định nghĩa và sử dụng giống hệt như các layer. Tuy nhiên, một model có một số chức năng bổ sung, bao gồm tất nhiên là các phương thức `compile()`, `fit()`, `evaluate()` và `predict()` (và một vài biến thể), cộng với phương thức `get_layers()` (có thể trả về bất kỳ layer nào của model theo tên hoặc theo chỉ mục) và phương thức `save()` (và hỗ trợ cho `tf.keras.models.load_model()` và `tf.keras.models.clone_model()`).

MẸO

Nếu mô hình cung cấp nhiều chức năng hơn các lớp, tại sao không định nghĩa mỗi lớp là một mô hình? Về mặt kỹ thuật, bạn có thể làm vậy, nhưng thường thì sẽ gọn gàng hơn nếu phân biệt các thành phần bên trong của mô hình (tức là các lớp hoặc các khối lớp có thể tái sử dụng) với chính mô hình (tức là đối tượng bạn sẽ huấn luyện). Các thành phần bên trong nên kế thừa từ lớp `Layer`, trong khi chính mô hình nên kế thừa từ lớp `Model`.

Với điều đó, bạn có thể xây dựng một cách tự nhiên và ngắn gọn hầu hết mọi mô hình mà bạn tìm thấy trong một bài báo, bằng cách sử dụng API Tuần tự, API Chức năng, API Phân lớp con, hoặc thậm chí là sự kết hợp của chúng. "Hầu hết" mọi mô hình? Vâng, vẫn còn một vài điều chúng ta cần xem xét: thứ nhất, làm thế nào để định nghĩa các hàm mất mát hoặc số liệu dựa trên các thành phần bên trong của mô hình, và thứ hai, làm thế nào để xây dựng một vòng lặp huấn luyện tùy chỉnh.

Tổn thất và các chỉ số dựa trên cấu trúc bên trong mô hình

Các hàm mất mát và chỉ số tùy chỉnh mà chúng ta đã định nghĩa trước đó đều dựa trên nhãn và dự đoán (và tùy chọn trọng số mẫu). Sẽ có lúc bạn muốn định nghĩa các hàm mất mát dựa trên các phần khác của mô hình, chẳng hạn như trọng số hoặc giá trị kích hoạt của các lớp ẩn. Điều này có thể hữu ích cho mục đích điều chỉnh hoặc để theo dõi một số khía cạnh nội bộ của mô hình.

Để định nghĩa một hàm mất mát tùy chỉnh dựa trên các thành phần bên trong của mô hình, hãy tính toán nó dựa trên bất kỳ phần nào của mô hình mà bạn muốn, sau đó truyền kết quả cho phương thức `add_loss()`. Ví dụ, hãy xây dựng một mô hình MLP hồi quy tùy chỉnh bao gồm một chồng năm lớp ẩn cộng với một lớp đầu ra. Mô hình tùy chỉnh này cũng sẽ có một đầu ra phụ trợ ở trên cùng của lớp ẩn trên cùng. Hàm mất mát liên quan đến đầu ra phụ trợ này sẽ được gọi là hàm mất mát tái tạo (xem [Chương 17](#)): đó là sự khác biệt bình phương trung bình giữa kết quả tái tạo và đầu vào. Bằng cách thêm hàm mất mát tái tạo này vào hàm mất mát chính, chúng ta sẽ khuyến khích mô hình bảo tồn càng nhiều thông tin càng tốt thông qua các lớp ẩn – ngay cả thông tin không trực tiếp hữu ích cho nhiệm vụ hồi quy. Trên thực tế, hàm mất mát này đôi khi cải thiện

Tổng quát hóa (đây là một hàm mất mát điều chỉnh). Cũng có thể thêm một chỉ số tùy chỉnh bằng cách sử dụng phương thức `add_metric()` của mô hình. Sau đây là mã cho mô hình tùy chỉnh này với hàm mất mát tái tạo tùy chỉnh và chỉ số tương ứng:

```
class ReconstructingRegressor(tf.keras.Model): def
    __init__(self, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.hidden = [tf.keras.layers.Dense(30,
activation="relu",

kernel_initializer="he_normal") cho
        - trong phạm
vi(5)] self.out = tf.keras.layers.Dense(output_dim)
self.reconstruction_mean =
    tf.keras.metrics.Mean( name="reconstruction_error")

    def build(self, batch_input_shape):
        n_inputs = batch_input_shape[-1]
        self.reconstruct = tf.keras.layers.Dense(n_inputs)

    def call(self, inputs, training=False): Z =
        inputs for
        layer in self.hidden: Z =
            layer(Z)
        reconstruction = self.reconstruct(Z)
        recon_loss = tf.reduce_mean(tf.square(reconstruction - inputs))

self.add_loss(0.05 * recon_loss) if training:

        result = self.reconstruction_mean(recon_loss)
        self.add_metric(result)
    return self.out(Z)
```

Hãy cùng xem xét đoạn mã này:

- Hàm khởi tạo tạo ra mạng nơ-ron sâu (DNN) với năm lớp ẩn dày đặc và một lớp đầu ra dày đặc. Chúng ta cũng tạo ra một chỉ số đo lường trung bình (Mean streaming metric) để theo dõi lỗi tái tạo trong quá trình huấn luyện.
- Phương thức `build()` tạo ra một lớp dày đặc bổ sung sẽ được sử dụng để tái tạo các đầu vào của mô hình. Nó phải được tạo ra ở đây vì

Số lượng đơn vị của nó phải bằng số lượng đầu vào, và con số này chưa được biết trước khi phương thức `build()` được gọi. 12

- Phương thức `call()` xử lý các đầu vào thông qua cả năm lớp ẩn, sau đó truyền kết quả qua lớp tái tạo, lớp này sẽ tạo ra hình ảnh tái tạo.
- Sau đó, phương thức `call()` tính toán tổn thất tái tạo (sai số bình phương trung bình giữa kết quả tái tạo và kết quả đầu vào), và thêm nó vào danh sách các tổn thất của mô hình bằng phương thức `add_loss()`. 13
 Lưu ý rằng chúng ta giảm tổn thất tái tạo bằng cách nhân nó với 0,05 (đây là một siêu tham số bạn có thể điều chỉnh). Điều này đảm bảo rằng tổn thất tái tạo không chi phối tổn thất chính.
- Tiếp theo, chỉ trong quá trình huấn luyện, phương thức `call()` cập nhật chỉ số tái tạo và thêm nó vào mô hình để có thể hiển thị.
 Ví dụ mã này thực ra có thể được đơn giản hóa bằng cách gọi `self.add_metric(recon_loss)` thay vì: Keras sẽ tự động theo dõi giá trị trung bình cho bạn (không cần `self.reconstruction_mean`).
- Cuối cùng, phương thức `call()` truyền đầu ra của các lớp ẩn đến lớp đầu ra và trả về đầu ra của nó.

Cả tổng thiệt hại và thiệt hại trong quá trình tái thiết đều sẽ giảm xuống trong quá trình huấn luyện:

Kỷ nguyên 1/5

363/363 [=====] - 1 giây 820us/bước - tổn thất: 0.7640 - lỗi tái tạo: 1.2728

Kỷ nguyên 2/5 363/363 [=====] - 0 giây

809us/bước -

tổn thất: 0.4584 - lỗi tái tạo: 0.6340 [...]

Trong hầu hết các trường hợp, tất cả những gì chúng ta đã thảo luận cho đến nay sẽ đủ để triển khai bất kỳ mô hình nào bạn muốn xây dựng, ngay cả với các kiến trúc, hàm mất mát và số liệu phức tạp. Tuy nhiên, đối với một số kiến trúc như GAN (xem [Chương 17](#)), bạn sẽ phải tùy chỉnh vòng lặp huấn luyện.

Trước khi đến đó, chúng ta phải xem xét cách tính đạo hàm.

Tự động trong TensorFlow.

Tính toán độ dốc bằng phương pháp tự động vi phân

Để hiểu cách sử dụng autodiff (xem [Chương 10](#) và [Phụ lục B](#))

Tính toán đạo hàm tự động, hãy xem xét một hàm ví dụ đơn giản:

```
def f(w1, w2):
    return 3 * w1 ** 2 + 2 * w1 * w2
```

Nếu bạn biết giải tích, bạn có thể tìm ra đạo hàm riêng của

Hàm này liên quan đến w_1 là $6w_2$. Bạn cũng có thể tìm thấy

Đạo hàm riêng của nó đối với w_2 là $2w_1$. Ví dụ, tại

điểm $(w_1, w_2) = (5, 3)$, các đạo hàm riêng này bằng 36 và

lần lượt là 10, vì vậy vectơ độ dốc tại điểm này là $(36, 10)$. Nhưng nếu điều này

Nếu đó là một mạng nơ-ron, chức năng sẽ phức tạp hơn nhiều.

thường có hàng chục nghìn tham số, và việc tìm ra một phần

Việc tính toán đạo hàm bằng phương pháp phân tích thủ công gần như là bất khả thi.

Giải pháp có thể là tính toán giá trị xấp xỉ của từng đạo hàm riêng bằng cách...

Đo lường mức độ thay đổi của kết quả đầu ra của hàm khi bạn điều chỉnh...

tham số tương ứng với một lượng rất nhỏ:

```
>>> w1, w2 = 5, 3
>>> eps = 1e-6
>>> (f(w1 + eps, w2) - f(w1, w2)) / eps
36.000003007075065
>>> (f(w1, w2 + eps) - f(w1, w2)) / eps
10.000000003174137
```

Có vẻ đúng rồi! Cách này hoạt động khá tốt và dễ thực hiện, nhưng nó...

Đây chỉ là một ước tính, và điều quan trọng là bạn cần gọi hàm $f()$ ít nhất một lần.

mỗi tham số (không phải hai lần, vì chúng ta có thể tính $f(w_1, w_2)$ chỉ một lần).

Việc phải gọi hàm $f()$ ít nhất một lần cho mỗi tham số khiến phương pháp này trở nên khó thực hiện.

Điều này quá khó khăn đối với các mạng nơ-ron lớn. Vì vậy, thay vào đó, chúng ta nên sử dụng phương

pháp tự động tính đạo hàm ngược. TensorFlow giúp việc này trở nên khá đơn giản:

```
w1, w2 = tf.Variable(5.), tf.Variable(3.) with
tf.GradientTape() as tape: z = f(w1, w2)
```

```
độ dốc = tape.gradient(z, [w1, w2])
```

Đầu tiên, chúng ta định nghĩa hai biến `w1` và `w2`, sau đó tạo một ngữ cảnh `tf.GradientTape` sẽ tự động ghi lại mọi thao tác liên quan đến một biến, và cuối cùng chúng ta yêu cầu ngữ cảnh này tính toán đạo hàm của kết quả `z` đối với cả hai biến `[w1, w2]`. Hãy cùng xem các đạo hàm mà TensorFlow đã tính toán:

```
>>> gradient
[<tf.Tensor: shape=(), dtype=float32, numpy=36.0>, <tf.Tensor:
  shape=(), dtype=float32, numpy=10.0>]
```

Tuyệt vời! Kết quả không chỉ chính xác (độ chính xác chỉ bị giới hạn bởi sai số dấu phẩy động), mà phương thức `gradient()` chỉ thực hiện các phép tính đã ghi lại một lần (theo thứ tự ngược lại), bất kể có bao nhiêu biến, vì vậy nó cực kỳ hiệu quả. Thật kỳ diệu!

MẸO

Để tiết kiệm bộ nhớ, chỉ nên đặt đoạn mã tối thiểu cần thiết vào bên trong khối `tf.GradientTape()`. Hoặc, tạm dừng ghi bằng cách tạo một khối `tape.stop_recording()` bên trong khối `tf.GradientTape()`.

Bảng từ sẽ tự động bị xóa ngay sau khi bạn gọi phương thức `gradient()` của nó, vì vậy bạn sẽ nhận được một ngoại lệ nếu cố gắng gọi `gradient()` hai lần:

```
với tf.GradientTape() là tape: z = f(w1,
    w2)

dz_dw1 = tape.gradient(z, w1) # trả về tensor 36.0 dz_dw2 =
tape.gradient(z, w2) # gây ra lỗi RuntimeError!
```

Nếu bạn cần gọi hàm `gradient()` nhiều hơn một lần, bạn phải lưu trữ đối tượng `tape` và xóa nó mỗi khi sử dụng xong để giải phóng tài nguyên: 14

```
với tf.GradientTape(persistent=True) as tape: z = f(w1, w2)
```

```
dz_dw1 = tape.gradient(z, w1) # trả về tensor 36.0 dz_dw2 = tape.gradient(z, w2) # trả về tensor
10.0, hoạt động tốt rồi! del tape
```

Theo mặc định, băng từ chỉ theo dõi các phép toán liên quan đến biến, vì vậy nếu bạn cố gắng tính đạo hàm của `z` đối với bất kỳ thứ gì khác ngoài biến, kết quả sẽ là `None`:

```
c1, c2 = tf.constant(5.), tf.constant(3.) with
tf.GradientTape() as tape: z = f(c1, c2)
```

```
gradients = tape.gradient(z, [c1, c2]) # trả về [None, None]
```

Tuy nhiên, bạn có thể buộc băng ghi theo dõi bất kỳ tensor nào bạn muốn, để ghi lại mọi thao tác liên quan đến chúng. Sau đó, bạn có thể tính toán đạo hàm đối với các tensor này, như thể chúng là các biến:

```
với tf.GradientTape() là tape:
    tape.watch(c1)
    tape.watch(c2) z
    = f(c1, c2)
```

```
gradients = tape.gradient(z, [c1, c2]) # trả về [tensor 36., tensor 10.]
```

Điều này có thể hữu ích trong một số trường hợp, chẳng hạn như nếu bạn muốn triển khai một hàm mất mát điều chỉnh (regularization loss) để phạt các giá trị kích hoạt thay đổi nhiều khi đầu vào thay đổi ít: hàm mất mát sẽ dựa trên độ dốc của các giá trị kích hoạt đối với đầu vào. Vì đầu vào không phải là biến, bạn cần phải hướng dẫn hệ thống theo dõi chúng.

Hầu hết thời gian, gradient tape được sử dụng để tính toán đạo hàm của một giá trị duy nhất (thường là hàm mất mát) đối với một tập hợp các giá trị (thường là các tham số của mô hình). Đây là lúc autodiff chế độ ngược phát huy tác dụng, vì nó chỉ cần thực hiện một lượt truyền thuận và một lượt truyền ngược để có được tất cả các đạo hàm cùng một lúc. Nếu bạn cố gắng tính toán đạo hàm của một vectơ, ví dụ như một vectơ chứa nhiều hàm mất mát, thì TensorFlow sẽ tính toán đạo hàm của tổng các phần tử trong vectơ đó. Vì vậy, nếu bạn cần lấy các đạo hàm riêng lẻ (ví dụ: đạo hàm của từng hàm mất mát đối với các tham số của mô hình), bạn phải gọi phương thức `jacobian()` của tape: nó sẽ thực hiện autodiff chế độ ngược một lần cho mỗi hàm mất mát trong vectơ (tất cả đều song song theo mặc định). Thậm chí có thể tính toán các đạo hàm riêng bậc hai (ma trận Hessian, tức là đạo hàm riêng của các đạo hàm riêng), nhưng điều này hiếm khi cần thiết trong thực tế (xem phần "Tính toán đạo hàm bằng Autodiff" trong notebook để biết ví dụ).

Trong một số trường hợp, bạn có thể muốn ngăn chặn sự lan truyền ngược của gradient qua một phần nào đó của mạng nơ-ron. Để làm điều này, bạn phải sử dụng hàm `tf.stop_gradient()`. Hàm này trả về các giá trị đầu vào trong quá trình truyền tiến (giống như `tf.identity()`), nhưng nó không cho phép gradient truyền qua trong quá trình lan truyền ngược (nó hoạt động như một hằng số):

```
def f(w1, w2):
    return 3 * w1 ** 2 + tf.stop_gradient(2 * w1 * w2)

with tf.GradientTape() as tape: z = f(w1,
    w2) # quá trình truyền tiến không bị ảnh hưởng bởi stop_gradient()

gradients = tape.gradient(z, [w1, w2]) # trả về [tensor 30., None]
```

Cuối cùng, đôi khi bạn có thể gặp một số vấn đề về số học khi tính toán đạo hàm. Ví dụ, nếu bạn tính đạo hàm của hàm căn bậc hai tại $x = 10$, độ dốc tại điểm đó không phải là vô hạn, $^{-50}$, kết quả sẽ là vô hạn. Trên thực tế, nhưng nó lớn hơn khả năng xử lý của số thực 32 bit:

```
>>> x = tf.Variable(1e-50) >>>
with tf.GradientTape() as tape:
```

```

...         z = tf.sqrt(x)
...
>>> tape.gradient(z, [x])
[<tf.Tensor: shape=(), dtype=float32, numpy=inf>]

```

Để giải quyết vấn đề này, thường thì nên cộng thêm một giá trị nhỏ vào x (chẳng hạn 10^{-6} như 10) khi tính căn bậc hai của nó.

Hàm mũ cũng thường gây đau đầu vì nó tăng cực nhanh. Ví dụ, cách định nghĩa hàm `my_softplus()` trước đó không ổn định về mặt số học. Nếu bạn tính `my_softplus(100.0)`, bạn sẽ nhận được vô cực thay vì kết quả chính xác (khoảng 100). Nhưng có thể viết lại hàm để làm cho nó ổn định về mặt số học: hàm `softplus` được định nghĩa là $\log(1 + \exp(z))$, cũng bằng $\log(1 + \exp(-|z|)) + \max(z, 0)$ (xem sổ tay để biết chứng minh toán học) và ưu điểm của dạng thứ hai này là số hạng mũ không thể bùng nổ.

Dưới đây là một cách triển khai tốt hơn cho hàm `my_softplus()`:

```

def my_softplus(z):
    return tf.math.log(1 + tf.exp(-tf.abs(z))) + tf.maximum(0.,
z)

```

Trong một số trường hợp hiếm hoi, một hàm ổn định về mặt số học vẫn có thể có đạo hàm không ổn định về mặt số học. Trong những trường hợp như vậy, bạn sẽ phải cho TensorFlow biết phương trình nào cần sử dụng cho đạo hàm, thay vì để nó sử dụng phương pháp tự động tính đạo hàm. Để làm điều này, bạn phải sử dụng decorator `@tf.custom_gradient` khi định nghĩa hàm và trả về cả kết quả thông thường của hàm cộng với một hàm tính toán đạo hàm. Ví dụ, hãy cập nhật hàm `my_softplus()` để cũng trả về một hàm đạo hàm ổn định về mặt số học:

```

@tf.custom_gradient def
my_softplus(z): def
    my_softplus_gradients(grads): # grads = lan truyền ngược từ các lớp trên return grads
    * (1 - 1 / (1 +
        tf.exp(z))) # grads ổn định của softplus

    kết quả = tf.math.log(1 + tf.exp(-tf.abs(z))) + tf.maximum(0.,

```

z)

trả về kết quả, `my_softplus_gradients`

Nếu bạn biết phép tính vi phân (xem sổ tay hướng dẫn về chủ đề này), bạn có thể thấy rằng đạo hàm của $\log(1 + \exp(z))$ là $\exp(z) / (1 + \exp(z))$.

Nhưng dạng này không ổn định: với các giá trị lớn của z , nó sẽ tính ra vô cực chia cho vô cực, trả về NaN. Tuy nhiên, với một chút biến đổi đại số, bạn có thể chứng minh rằng nó cũng bằng $1 - 1 / (1 + \exp(z))$, vốn ổn định. Hàm `my_softplus_gradients()` sử dụng phương trình này để tính toán đạo hàm. Lưu ý rằng hàm này sẽ nhận đầu vào là các đạo hàm đã được lan truyền ngược cho đến hàm `my_softplus()`; và theo quy tắc chuỗi, chúng ta phải nhân chúng với các đạo hàm của hàm này.

Giờ đây, khi tính toán đạo hàm của hàm `my_softplus()`, chúng ta nhận được kết quả chính xác, ngay cả với các giá trị đầu vào lớn.

Chúc mừng! Giờ đây bạn có thể tính toán đạo hàm của bất kỳ hàm nào (miễn là hàm đó khả vi tại điểm bạn tính toán), thậm chí cả lan truyền ngược theo khối khi cần thiết, và tự viết các hàm đạo hàm của riêng mình! Điều này có lẽ mang lại sự linh hoạt hơn mức bạn cần, ngay cả khi bạn tự xây dựng các vòng lặp huấn luyện tùy chỉnh, như chúng ta sẽ thấy ngay bây giờ.

Các vòng lặp huấn luyện tùy chỉnh

Trong một số trường hợp, phương thức `fit()` có thể không đủ linh hoạt cho những gì bạn cần làm. Ví dụ, bài báo [Wide & Deep](#) Như chúng ta đã thảo luận trong [Chương 10](#), nó sử dụng hai trình tối ưu hóa khác nhau: một cho đường dẫn rộng và một cho đường dẫn sâu. Vì phương thức `fit()` chỉ sử dụng một trình tối ưu hóa (trình tối ưu hóa mà chúng ta chỉ định khi biên dịch mô hình), việc triển khai bài báo này yêu cầu bạn phải tự viết vòng lặp tùy chỉnh của riêng mình.

Bạn cũng có thể muốn viết các vòng lặp huấn luyện tùy chỉnh đơn giản để cảm thấy tự tin hơn rằng chúng thực hiện chính xác những gì bạn dự định (có thể bạn không chắc chắn về một số chi tiết của phương thức `fit()`). Đôi khi, việc làm cho mọi thứ rõ ràng có thể khiến bạn cảm thấy an toàn hơn. Tuy nhiên, hãy nhớ rằng việc viết một

Vòng lặp huấn luyện tùy chỉnh sẽ làm cho mã của bạn dài hơn, dễ xảy ra lỗi hơn và khó bảo trì hơn.

MẸO

Trừ khi bạn đang học hoặc thực sự cần sự linh hoạt hơn, bạn nên ưu tiên sử dụng phương thức `fit()` thay vì tự triển khai vòng lặp huấn luyện của riêng mình, đặc biệt nếu bạn làm việc trong một nhóm.

Trước tiên, hãy xây dựng một mô hình đơn giản. Không cần biên dịch nó, vì chúng ta sẽ tự xử lý vòng lặp huấn luyện:

```
l2_reg = tf.keras.regularizers.l2(0.05) model =
tf.keras.models.Sequential([ tf.keras.layers.Dense(30,
    activation="relu", kernel_initializer="he_normal",
                                kernel_regularizer=l2_reg),
    tf.keras.layers.Dense(1, kernel_regularizer=l2_reg)
])
```

Tiếp theo, chúng ta hãy tạo một hàm nhỏ để lấy mẫu ngẫu nhiên một nhóm các trường hợp từ tập dữ liệu huấn luyện (trong [Chương 13](#), chúng ta sẽ thảo luận về API Dữ liệu, cung cấp một giải pháp thay thế tốt hơn nhiều):

```
def random_batch(X, y, batch_size=32): idx =
    np.random.randint(len(X), size=batch_size) return X[idx], y[idx]
```

Chúng ta cũng cần định nghĩa một hàm để hiển thị trạng thái huấn luyện, bao gồm số bước, tổng số bước, tổn thất trung bình kể từ khi bắt đầu epoch (tức là chúng ta sẽ sử dụng chỉ số Mean để tính toán), và các chỉ số khác:

```
def print_status_bar(step, total, loss, metrics=None): metrics = -
    ".join([f"{m.name}: {m.result():.4f}" for m in [loss] + (metrics or
        []))
    end = "" if step < total else "\n"
    print(f"\r{step}/{total} - + metrics, end=end)
```

Đoạn mã này khá dễ hiểu, trừ khi bạn chưa quen với định dạng chuỗi trong Python:

`{m.result():.4f}` sẽ định dạng kết quả của chỉ số dưới dạng số thực với bốn chữ số sau dấu thập phân; hơn nữa, việc sử dụng `\r` (ký tự xuống dòng) cùng với `end=""` đảm bảo rằng thanh trạng thái luôn được in trên cùng một dòng.

Vậy thì, chúng ta bắt tay vào việc thôi! Đầu tiên, chúng ta cần định nghĩa một số siêu tham số và chọn bộ tối ưu hóa, hàm mất mát và các chỉ số đánh giá (chỉ có MAE trong ví dụ này):

```
n_epochs = 5
batch_size = 32
n_steps = len(X_train) // batch_size
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
loss_fn = tf.keras.losses.mean_squared_error
mean_loss = tf.keras.metrics.Mean(name="mean_loss")
metrics = [tf.keras.metrics.MeanAbsoluteError()]
```

Và giờ chúng ta đã sẵn sàng để xây dựng hệ thống vòng lặp tùy chỉnh!

```
for epoch in range(1, n_epochs + 1):
    print("Epoch {} / {}".format(epoch, n_epochs))
    for step in range(1, n_steps + 1):
        X_batch, y_batch = random_batch(X_train_scaled, y_train)
        with tf.GradientTape() as tape:
            y_pred = model(X_batch, training=True)
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)

        độ dốc = bằng.độ dốc(mất mát, model.trainable_variables)
        optimizer.apply_gradients(zip(gradient, model.trainable_variables))

        mean_loss(loss)
        for metric in metrics:
            metric(y_batch, y_pred)

        print_status_bar(step, n_steps, mean_loss, metrics)

    for metric in [mean_loss] + metrics:
        metric.reset_states()
```

Đoạn mã này khá phức tạp, vì vậy chúng ta hãy cùng phân tích từng phần:

- Chúng ta tạo ra hai vòng lặp lồng nhau: một cho các epoch, vòng lặp còn lại cho các batch trong một epoch.
- Sau đó, chúng ta chọn ngẫu nhiên một lô dữ liệu từ tập dữ liệu huấn luyện.
- Bên trong khối ``tf.GradientTape()``, chúng ta thực hiện dự đoán cho một batch, sử dụng mô hình như một hàm, và tính toán tổn thất: nó bằng tổn thất chính cộng với các tổn thất khác (trong mô hình này, có một tổn thất điều chỉnh cho mỗi lớp). Vì hàm ``mean_squared_error()`` trả về một tổn thất cho mỗi trường hợp, chúng ta tính toán giá trị trung bình trên toàn bộ batch bằng cách sử dụng ``tf.reduce_mean()`` (nếu bạn muốn áp dụng các trọng số khác nhau cho mỗi trường hợp, đây là nơi bạn sẽ thực hiện). Các tổn thất điều chỉnh đã được giảm xuống thành một giá trị vô hướng duy nhất, vì vậy chúng ta chỉ cần cộng chúng lại (sử dụng ``tf.add_n()``, hàm này cộng nhiều tensor có cùng hình dạng và kiểu dữ liệu).
- Tiếp theo, chúng ta yêu cầu bằng ghi tính toán độ dốc của hàm mất mát đối với từng biến có thể huấn luyện được—không phải tất cả các biến!—và áp dụng chúng vào bộ tối ưu hóa để thực hiện bước Giảm độ dốc (Gradient Descent).
- Sau đó, chúng ta cập nhật giá trị trung bình của tổn thất và các chỉ số (trong suốt epoch hiện tại), rồi hiển thị thanh trạng thái.
- Vào cuối mỗi kỷ nguyên, chúng ta đặt lại trạng thái của tổn thất trung bình và các chỉ số đo lường.

Nếu bạn muốn áp dụng Gradient Clipping (xem [Chương 11](#)), chỉ cần đặt siêu tham số `clipnorm` hoặc `clipvalue` của trình tối ưu hóa. Nếu bạn muốn áp dụng bất kỳ phép biến đổi nào khác cho gradient, chỉ cần thực hiện điều đó trước khi gọi phương thức `apply_gradients()`. Và nếu bạn muốn thêm các ràng buộc trọng số vào mô hình của mình (ví dụ: bằng cách đặt `kernel_constraint` hoặc `bias_constraint` khi tạo một lớp), bạn nên cập nhật vòng lặp huấn luyện để áp dụng các ràng buộc này ngay sau khi gọi `apply_gradients()`, như sau:

```
cho biến trong model.variables:
    nếu biến.constraint không phải là None:
        biến.gán(biến.ràng buộc(biến))
```

CẢNH BÁO

Đừng quên đặt `training=True` khi gọi mô hình trong vòng lặp huấn luyện, đặc biệt nếu mô hình của bạn hoạt động khác nhau trong quá trình huấn luyện và kiểm thử (ví dụ: nếu nó sử dụng BatchNormalization hoặc Dropout). Nếu đó là mô hình tùy chỉnh, hãy đảm bảo truyền tham số `training` đến các lớp mà mô hình của bạn gọi.

Như bạn thấy đây, có khá nhiều thứ cần phải làm cho đúng, và rất dễ mắc sai lầm. Nhưng mặt khác, bạn có toàn quyền kiểm soát.

Giờ bạn đã biết cách tùy chỉnh bất kỳ phần nào của mô hình và thuật toán huấn luyện¹⁵, hãy cùng xem cách bạn có thể sử dụng tính năng tạo đồ thị tự động của TensorFlow: nó có thể tăng tốc đáng kể mã tùy chỉnh của bạn và cũng giúp mã đó có thể tương thích với bất kỳ nền tảng nào được TensorFlow hỗ trợ (xem [Chương 19](#)).

Các hàm và đồ thị trong TensorFlow

Trở lại thời TensorFlow 1, đồ thị là điều không thể tránh khỏi (cũng như những phức tạp đi kèm với chúng) vì chúng là một phần cốt lõi của API của TensorFlow.

Từ phiên bản TensorFlow 2 (phát hành năm 2019), đồ thị vẫn còn đó, nhưng không còn đóng vai trò trung tâm như trước, và chúng dễ sử dụng hơn rất nhiều (rất nhiều!). Để chứng minh điều đó, hãy bắt đầu với một hàm đơn giản tính lập phương của đầu vào:

```
def cube(x): return x ** 3
```

Rõ ràng chúng ta có thể gọi hàm này với một giá trị Python, chẳng hạn như số nguyên hoặc số thực, hoặc chúng ta có thể gọi nó với một tensor:

```
>>> khối lập phương(2)
8
```

```
>>> cube(tf.constant(2.0)) <tf.Tensor:
shape=(), dtype=float32, numpy=8.0>
```

Bây giờ, hãy sử dụng `tf.function()` để chuyển đổi hàm Python này thành một hàm TensorFlow:

```
>>> tf_cube = tf.function(cube) >>> tf_cube

<tensorflow.python.eager.def_function.Function at 0x7fbfe0c54d50>
```

Hàm TF này sau đó có thể được sử dụng chính xác như hàm Python gốc và sẽ trả về cùng một kết quả (nhưng luôn luôn dưới dạng tensor):

```
>>> tf_cube(2)
<tf.Tensor: shape=(), dtype=int32, numpy=8> >>>
tf_cube(tf.constant(2.0)) <tf.Tensor:
shape=(), dtype=float32, numpy=8.0>
```

Về mặt kỹ thuật, `tf.function()` đã phân tích các phép tính được thực hiện bởi hàm `cube()` và tạo ra một đồ thị tính toán tương đương!

Như bạn thấy đây, quá trình này khá dễ dàng (chúng ta sẽ xem kết quả cụ thể hơn sau). Ngoài ra, chúng ta cũng có thể sử dụng `tf.function` như một decorator; cách này thực tế phổ biến hơn:

```
@tf.function def
tf_cube(x):
    trả về x ** 3
```

Hàm Python gốc vẫn có sẵn thông qua thuộc tính `python_function` của hàm TF, trong trường hợp bạn cần đến nó:

```
>>> tf_cube.python_function(2) 8
```

TensorFlow tối ưu hóa đồ thị tính toán, loại bỏ các nút không sử dụng, đơn giản hóa các biểu thức (ví dụ: $1 + 2$ sẽ được thay thế bằng 3), và nhiều hơn nữa. Sau khi đồ thị được tối ưu hóa sẵn sàng, hàm TF sẽ thực thi các thao tác trên đồ thị một cách hiệu quả, theo đúng thứ tự (và song song khi cần thiết).

(có thể). Do đó, một hàm TF thường sẽ chạy nhanh hơn nhiều so với hàm Python gốc, đặc biệt nếu nó thực hiện các phép tính phức tạp.

Hầu hết thời gian bạn sẽ không cần biết nhiều hơn thế: khi muốn tăng hiệu năng của một hàm Python, chỉ cần chuyển đổi nó thành một hàm TF. Vậy thôi!

Hơn nữa, nếu bạn đặt `jit_compile=True` khi gọi `tf.function()`, thì TensorFlow sẽ sử dụng Đại số tuyến tính tăng tốc (XLA) để biên dịch các kernel chuyên dụng cho đồ thị của bạn, thường kết hợp nhiều phép toán. Ví dụ, nếu hàm TF của bạn gọi `tf.reduce_sum(a * b + c)`, thì nếu không có XLA, hàm đó trước tiên cần tính ab và lưu kết quả vào một biến tạm thời, sau đó cộng c vào biến đó, và cuối cùng gọi `tf.reduce_sum()` trên kết quả. Với XLA, toàn bộ quá trình tính toán được biên dịch thành một kernel duy nhất, kernel này sẽ tính toán `tf.reduce_sum(a * b + c)` trong một lần, mà không cần sử dụng bất kỳ biến tạm thời lớn nào. Điều này không chỉ nhanh hơn nhiều mà còn sử dụng ít RAM hơn đáng kể.

Khi bạn viết một hàm mất mát tùy chỉnh, một chỉ số tùy chỉnh, một lớp tùy chỉnh, hoặc bất kỳ hàm tùy chỉnh nào khác và sử dụng nó trong mô hình Keras (như chúng ta đã làm trong suốt chương này), Keras sẽ tự động chuyển đổi hàm của bạn thành một hàm TF—không cần sử dụng `tf.function()`. Vì vậy, hầu hết thời gian, quá trình này hoàn toàn minh bạch. Và nếu bạn muốn Keras sử dụng XLA, bạn chỉ cần đặt `jit_compile=True` khi gọi phương thức `compile()`. Dễ!

MẸO

Bạn có thể yêu cầu Keras không chuyển đổi các hàm Python của mình thành hàm TF bằng cách đặt `'dynamic=True'` khi tạo một lớp tùy chỉnh hoặc một mô hình tùy chỉnh. Ngoài ra, bạn có thể đặt `'run_eagerly=True'` khi gọi phương thức `'compile()'` của mô hình.

Theo mặc định, một hàm TF sẽ tạo ra một đồ thị mới cho mỗi tập hợp hình dạng và kiểu dữ liệu đầu vào duy nhất và lưu trữ đồ thị đó để sử dụng cho các lần gọi tiếp theo. Ví dụ:

Nếu bạn gọi `tf_cube(tf.constant(10))`, một đồ thị sẽ được tạo cho các tensor `int32` có hình dạng `[]`. Sau đó, nếu bạn gọi `tf_cube(tf.constant(20))`, đồ thị đó sẽ được sử dụng lại. Nhưng nếu sau đó bạn gọi `tf_cube(tf.constant([10, 20]))`, một đồ thị mới sẽ được tạo cho các tensor `int32` có hình dạng `[2]`. Đây là cách các hàm TF xử lý tính đa hình (tức là các kiểu và hình dạng đối số khác nhau). Tuy nhiên, điều này chỉ đúng với các đối số tensor: nếu bạn truyền các giá trị số Python cho một hàm TF, một đồ thị mới sẽ được tạo cho mỗi giá trị khác nhau: ví dụ, gọi `tf_cube(10)` và `tf_cube(20)` sẽ tạo ra hai đồ thị.

CẢNH BÁO

Nếu bạn gọi một hàm TF nhiều lần với các giá trị số Python khác nhau, thì nhiều đồ thị sẽ được tạo ra, làm chậm chương trình của bạn và tiêu tốn nhiều RAM (bạn phải xóa hàm TF để giải phóng nó). Các giá trị Python nên được dành riêng cho các đối số có ít giá trị duy nhất, chẳng hạn như các siêu tham số như số lượng neuron trên mỗi lớp. Điều này cho phép TensorFlow tối ưu hóa tốt hơn từng biến thể của mô hình của bạn.

AutoGraph và Tracing

Vậy TensorFlow tạo ra đồ thị như thế nào? Nó bắt đầu bằng cách phân tích mã nguồn của hàm Python để nắm bắt tất cả các câu lệnh điều khiển luồng, chẳng hạn như vòng lặp `for`, vòng lặp `while` và câu lệnh `if`, cũng như các câu lệnh `break`, `continue` và `return`. Bước đầu tiên này được gọi là AutoGraph. Lý do TensorFlow phải phân tích mã nguồn là vì Python không cung cấp bất kỳ cách nào khác để nắm bắt các câu lệnh điều khiển luồng: nó cung cấp các phương thức đặc biệt như `__add__()` và `__mul__()` để nắm bắt các toán tử như `+` và `*`, nhưng không có các phương thức đặc biệt như `__while__()` hoặc `__if__()`. Sau khi phân tích mã của hàm, AutoGraph sẽ xuất ra một phiên bản được nâng cấp của hàm đó, trong đó tất cả các câu lệnh điều khiển luồng được thay thế bằng các thao tác TensorFlow thích hợp, chẳng hạn như `tf.while_loop()` cho vòng lặp và `tf.cond()` cho câu lệnh `if`. Ví dụ, trong [Hình 12-4](#), AutoGraph phân tích mã nguồn của hàm...

Hàm `sum_squares()` của Python tạo ra hàm `tf__sum_squares()`. Trong hàm này, vòng lặp `for` được thay thế bằng định nghĩa của hàm `loop_body()` (chứa phần thân của vòng lặp `for` ban đầu), tiếp theo là lời gọi đến hàm `for_stmt()`. Lời gọi này sẽ xây dựng phép toán `tf.while_loop()` thích hợp trong đồ thị tính toán.

```
@tf.function
def sum_squares(n):
    s = 0
    for i in tf.range(n + 1):
        s += i ** 2
    return s
```

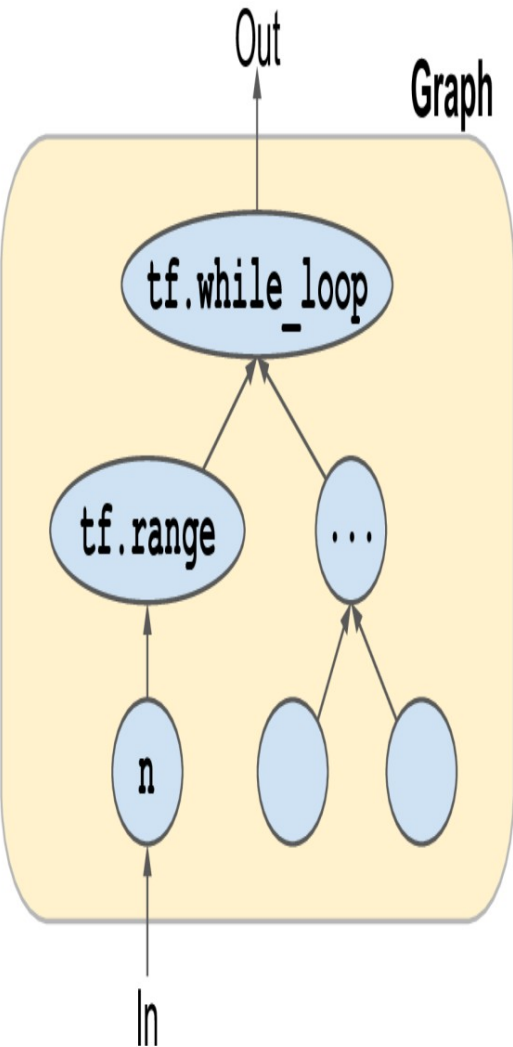
1. AutoGraph

Tensor (name="n:0",
shape=(),
dtype=int32)

```
def tf_sum_squares(n):
    s = 0
    def loop_body(i, s):
        s += i ** 2
        return s,
    s, = ag_.for_stmt(...,
                      loop_body,
                      (s,))
    return s
```

(shortened)

2. Tracing



Hình 12-4. Cách TensorFlow tạo đồ thị bằng AutoGraph và tracing.

Tiếp theo, TensorFlow gọi hàm “được nâng cấp” này, nhưng thay vì truyền đối số, nó truyền một tensor tượng trưng—một tensor không có giá trị thực tế nào, chỉ có tên, kiểu dữ liệu và hình dạng. Ví dụ, nếu bạn gọi `sum_squares(tf.constant(10))`, thì hàm `tf__sum_squares()` sẽ được gọi với một tensor tượng trưng kiểu `int32` và hình dạng `[]`. Hàm sẽ chạy ở chế độ đồ thị, có nghĩa là mỗi thao tác TensorFlow sẽ thêm một nút vào đồ thị để biểu diễn chính nó và (các) tensor đầu ra của nó (trái ngược với chế độ thông thường, được gọi là thực thi tức thời, hoặc chế độ tức thời). Ở chế độ đồ thị, các thao tác TF không thực hiện bất kỳ phép tính nào. Chế độ đồ thị là chế độ mặc định trong TensorFlow 1. Trong [Hình 12-4](#), bạn có thể thấy hàm `tf__sum_squares()` được gọi với một tensor tượng trưng làm đối số của nó (trong trường hợp này, một tensor `int32` có hình dạng `[]`) và đồ thị cuối cùng được tạo ra trong quá trình theo dõi. Các nút biểu thị các phép toán, và các mũi tên biểu thị các tensor (cả hàm được tạo ra và đồ thị đều được đơn giản hóa).

MẸO

Để xem mã nguồn của hàm được tạo ra, bạn có thể gọi `tf.autograph.to_code(sum_squares.python_function)`. Mã này không nhằm mục đích đẹp mắt, nhưng đôi khi nó có thể hữu ích cho việc gỡ lỗi.

Quy tắc chức năng TF

Hầu hết các trường hợp, việc chuyển đổi một hàm Python thực hiện các phép toán TensorFlow thành một hàm TF rất đơn giản: chỉ cần trang trí nó bằng `@tf.function` hoặc để Keras tự động thực hiện. Tuy nhiên, có một vài quy tắc cần tuân thủ:

- Nếu bạn gọi bất kỳ thư viện bên ngoài nào, bao gồm NumPy hoặc thậm chí thư viện chuẩn, lệnh gọi này sẽ chỉ chạy trong quá trình theo dõi; nó sẽ không phải là một phần của đồ thị. Thực tế, một đồ thị TensorFlow chỉ có thể bao gồm các cấu trúc TensorFlow (tensor, phép toán, biến, tập dữ liệu, v.v.). Vì vậy,

Hãy đảm bảo bạn sử dụng `tf.reduce_sum()` thay vì `np.sum()`, `tf.sort()` thay vì hàm `sorted()` tích hợp sẵn, v.v. (trừ khi bạn thực sự muốn mã chỉ chạy trong quá trình theo dõi). Điều này có một vài hệ quả bổ sung:

- Nếu bạn định nghĩa một hàm TF $f(x)$ chỉ trả về `np.random.rand()`, thì một số ngẫu nhiên sẽ chỉ được tạo ra khi hàm được theo dõi, vì vậy `f(tf.constant(2.))` và `f(tf.constant(3.))` sẽ trả về cùng một số ngẫu nhiên, nhưng `f(tf.constant([2., 3.]))` sẽ trả về một số khác. Nếu bạn thay thế `np.random.rand()` bằng `tf.random.uniform([])`, thì một số ngẫu nhiên mới sẽ được tạo ra trong mỗi lần gọi, vì thao tác này sẽ là một phần của đồ thị.
- Nếu mã không phải TensorFlow của bạn có các tác dụng phụ (chẳng hạn như ghi nhật ký hoặc cập nhật bộ đếm Python), thì bạn không nên mong đợi các tác dụng phụ đó xảy ra mỗi khi bạn gọi hàm TF, vì chúng chỉ xảy ra khi hàm được theo dõi.
- Bạn có thể gói mã Python tùy ý trong một thao tác `tf.py_function()`, nhưng làm như vậy sẽ làm giảm hiệu suất, vì TensorFlow sẽ không thể thực hiện bất kỳ tối ưu hóa đồ thị nào trên mã này. Nó cũng sẽ làm giảm tính khả chuyển, vì đồ thị sẽ chỉ chạy trên các nền tảng có Python (và nơi các thư viện phù hợp được cài đặt).
- Bạn có thể gọi các hàm Python khác hoặc các hàm TF, nhưng chúng phải tuân theo các quy tắc tương tự, vì TensorFlow sẽ ghi lại các thao tác của chúng trong đồ thị tính toán. Lưu ý rằng các hàm khác này không cần được trang trí bằng `@tf.function`.
- Nếu hàm tạo một biến TensorFlow (hoặc bất kỳ đối tượng TensorFlow có trạng thái nào khác, chẳng hạn như tập dữ liệu hoặc hàng đợi), thì việc đó phải được thực hiện ngay trong lần gọi đầu tiên, và chỉ khi đó, nếu không bạn sẽ gặp ngoại lệ. Thông thường, nên tạo các biến bên ngoài hàm TF (ví dụ:

(trong phương thức `build()` của một lớp tùy chỉnh). Nếu bạn muốn gán một giá trị mới cho biến, hãy đảm bảo bạn gọi phương thức `assign()` của nó, thay vì sử dụng toán tử `=`.

- Mã nguồn của hàm Python của bạn phải được cung cấp cho TensorFlow. Nếu mã nguồn không có sẵn (ví dụ: nếu bạn định nghĩa hàm của mình trong Python shell, điều này không cho phép truy cập vào mã nguồn, hoặc nếu bạn chỉ triển khai các tệp Python `*.pyc` đã biên dịch lên môi trường sản xuất), thì quá trình tạo đồ thị sẽ thất bại hoặc hoạt động bị hạn chế.
- TensorFlow chỉ ghi lại các vòng lặp ``for`` lặp qua một tensor hoặc ``tf.data.Dataset`` (xem [Chương 13](#)). Vì vậy, hãy đảm bảo bạn sử dụng ``for i in tf.range(x)`` thay vì ``for i in range(x)``, nếu không vòng lặp sẽ không được ghi lại trong đồ thị. Thay vào đó, nó sẽ chạy trong quá trình theo dõi. (Đây có thể là điều bạn muốn nếu vòng lặp ``for`` được dùng để xây dựng đồ thị, ví dụ như để tạo từng lớp trong mạng nơ-ron.)
- Như thường lệ, vì lý do hiệu năng, bạn nên ưu tiên sử dụng phương pháp vector hóa bất cứ khi nào có thể, thay vì sử dụng vòng lặp.

Đã đến lúc tổng kết! Trong chương này, chúng ta bắt đầu với tổng quan ngắn gọn về TensorFlow, sau đó xem xét API cấp thấp của TensorFlow, bao gồm tensor, phép toán, biến và các cấu trúc dữ liệu đặc biệt. Tiếp theo, chúng ta đã sử dụng các công cụ này để tùy chỉnh hầu hết mọi thành phần trong `tf.keras`. Cuối cùng, chúng ta đã xem xét cách TF Functions có thể tăng hiệu suất, cách tạo đồ thị bằng `AutoGraph` và `tracing`, và những quy tắc cần tuân theo khi viết TF Functions (nếu bạn muốn tìm hiểu sâu hơn về các đồ thị được tạo ra, bạn sẽ tìm thấy chi tiết kỹ thuật trong [Phụ lục D](#)).

Trong chương tiếp theo, chúng ta sẽ xem xét cách tải và tiền xử lý dữ liệu hiệu quả bằng TensorFlow.

Bài tập

1. Bạn có thể mô tả TensorFlow bằng một câu ngắn gọn như thế nào? Các tính năng chính của nó là gì?
Bạn có thể kể tên một số thư viện Deep Learning phổ biến khác không?
2. TensorFlow có phải là một sự thay thế hoàn toàn cho NumPy không? Những điểm khác biệt chính giữa hai thư viện này là gì?
3. Bạn có nhận được kết quả giống nhau với `tf.range(10)` và `tf.constant(np.arange(10))` không?
4. Bạn có thể kể tên sáu cấu trúc dữ liệu khác có sẵn trong TensorFlow không?
Ngoài các tensor thông thường?
5. Có thể định nghĩa hàm mất mát tùy chỉnh bằng cách viết một hàm hoặc bằng cách...
Tạo lớp con của lớp `tf.keras.losses.Loss`. Trong trường hợp nào thì nên sử dụng mỗi tùy chọn?
6. Tương tự, một chỉ số tùy chỉnh có thể được định nghĩa trong một hàm hoặc một lớp con của `tf.keras.metrics.Metric`. Khi nào bạn sẽ sử dụng mỗi tùy chọn?
7. Khi nào nên tạo lớp tùy chỉnh thay vì mô hình tùy chỉnh?
8. Một số trường hợp sử dụng nào yêu cầu việc tự viết chương trình đào tạo tùy chỉnh?
vòng lặp?
9. Các thành phần Keras tùy chỉnh có thể chứa mã Python tùy ý hay không, hay bắt buộc phải như vậy?
Liệu chúng có thể chuyển đổi thành các hàm TF không?
10. Những quy tắc chính nào cần tuân thủ nếu bạn muốn chuyển đổi một hàm thành hàm TF?
11. Khi nào bạn cần tạo một mô hình Keras động? Bạn làm điều đó như thế nào?
Làm vậy sao? Tại sao không làm cho tất cả các mô hình của bạn trở nên năng động?
12. Triển khai một lớp tùy chỉnh thực hiện Chuẩn hóa lớp (chúng ta sẽ...)
(Sử dụng loại lớp này trong [Chương 15](#)):
 - a. Phương thức `build()` nên định nghĩa hai trọng số có thể huấn luyện α và β , cả hai đều có hình dạng `input_shape[-1:]` và kiểu dữ liệu

`tf.float32`. α nên được khởi tạo bằng 1 và β bằng 0.

- b. Phương thức `call()` nên tính toán giá trị trung bình μ và độ lệch chuẩn σ của các đặc trưng của mỗi trường hợp. Để làm điều này, bạn có thể sử dụng `tf.nn.moments(inputs, axes=-1, keepdims=True)`, hàm này trả về giá trị trung bình μ và phương sai σ của tất cả các trường hợp (tính căn bậc hai của phương sai để có được độ lệch chuẩn). Sau đó, hàm nên tính toán và trả về $\alpha (X - \mu)/(\sigma + \epsilon) + \beta$, trong đó biểu thị phép nhân từng phần tử ($*$) và ϵ là một hệ số làm mịn (hằng số nhỏ để tránh chia cho 0, ví dụ: 0.001).
- c. Đảm bảo rằng lớp tùy chỉnh của bạn tạo ra kết quả tương tự (hoặc gần như tương tự). (Tương tự) đầu ra như lớp `tf.keras.layers.LayerNormalization`.

13. Huấn luyện mô hình sử dụng vòng lặp huấn luyện tùy chỉnh để giải quyết bài toán Thời trang.

Bộ dữ liệu MNIST (xem [Chương 10](#)).

- a. Hiển thị epoch, iteration, mean training loss và mean độ chính xác qua từng kỷ nguyên (được cập nhật sau mỗi lần lặp), cũng như tổn thất và độ chính xác trên tập dữ liệu kiểm chứng vào cuối mỗi kỷ nguyên.
- b. Hãy thử sử dụng một trình tối ưu hóa khác với tốc độ học khác nhau. các lớp trên và các lớp dưới.

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://homl.info/colab3>.

1 Tuy nhiên, thư viện PyTorch của Facebook hiện đang phổ biến hơn trong giới học thuật: nhiều bài báo trích dẫn PyTorch hơn TensorFlow hoặc Keras. Hơn nữa, thư viện JAX của Google đang ngày càng được ưa chuộng, đặc biệt là trong giới học thuật.

2. TensorFlow bao gồm một API Học sâu khác gọi là API Estimators, nhưng hiện tại nó đã được...
Đã lỗi thời.

3. Nếu bạn cần (nhưng có lẽ bạn sẽ không cần), bạn có thể tự viết các thao tác của riêng mình bằng cách sử dụng...
API C++.

4. Để tìm hiểu thêm về TPU và cách chúng hoạt động, hãy truy cập <https://homl.info/tpus>.

5. Một ngoại lệ đáng chú ý là `tf.math.log()`, hàm này thường được sử dụng nhưng không có bí danh `tf.log()`, vì có thể bị nhầm lẫn với việc ghi nhật ký.
6. Không nên sử dụng trung bình có trọng số: nếu bạn làm vậy, thì hai trường hợp có cùng trọng số nhưng thuộc các lô khác nhau sẽ có tác động khác nhau đến quá trình huấn luyện, tùy thuộc vào tổng trọng số của mỗi lô.
7. Cú pháp `**x, [...]` được thêm vào trong Python 3.5, để hợp nhất tất cả các cặp khóa/giá trị từ từ điển `x` vào một từ điển khác. Từ Python 3.9 trở đi, bạn có thể sử dụng cú pháp `x | y` để sử dụng hơn (trong đó `x` và `y` là hai từ điển).
- 8 Tuy nhiên, tổn thất Huber hiếm khi được sử dụng làm thước đo-MAE hoặc MSE thường được sử dụng nhiều hơn. Ưu tiên.
9. Lớp này chỉ mang tính chất minh họa. Một cách triển khai đơn giản và tốt hơn là chỉ cần kế thừa lớp `tf.keras.metrics.Mean`; xem phần "Streaming metrics" của notebook để biết ví dụ.
10. API của Keras gọi tham số này là `input_shape`, nhưng vì nó cũng bao gồm cả batch. Tôi thích gọi kích thước đó là `batch_input_shape`.
11. Thuật ngữ "API kế thừa lớp con" trong Keras thường chỉ đề cập đến việc tạo các mô hình tùy chỉnh bằng cách kế thừa lớp con, mặc dù nhiều thứ khác cũng có thể được tạo ra bằng cách kế thừa lớp con, như chúng ta đã thấy trong chương này.
12. Do sự cố TensorFlow #46858, lệnh gọi `super().build()` có thể bị lỗi trong trường hợp này. trừ khi sự cố đã được khắc phục vào thời điểm bạn đọc bài này. Nếu không, bạn cần thay thế dòng này bằng `self.built = True`.
- 13 Bạn cũng có thể gọi `add_loss()` trên bất kỳ lớp nào bên trong mô hình, vì mô hình thực hiện đệ quy. Tổng hợp các khoản lỗ từ tất cả các tầng của nó.
14. Nếu đối tượng bằng từ nằm ngoài phạm vi, ví dụ như khi hàm sử dụng nó trả về, trình thu gom rác của Python sẽ tự động xóa nó.
15. Ngoại trừ các công cụ tối ưu hóa, vì rất ít người tùy chỉnh chúng; hãy xem phần "Tùy chỉnh". Xem phần "Optimizers" trong sổ tay hướng dẫn để biết ví dụ.
- 16 Tuy nhiên, trong ví dụ đơn giản này, đồ thị tính toán quá nhỏ nên không có gì để tối ưu hóa cả, do đó hàm `tf_cube()` thực tế chạy chậm hơn nhiều so với hàm `cube()`.

Chương 13. Tải và tiền xử lý dữ liệu với TensorFlow

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 13 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Trong **Chương 2**, chúng ta đã thấy rằng việc tải và tiền xử lý dữ liệu là một phần quan trọng của bất kỳ dự án Học máy nào. Chúng ta đã sử dụng Pandas để tải và khám phá tập dữ liệu nhà ở California (đã được chỉnh sửa) – được lưu trữ trong tệp CSV – và chúng ta đã áp dụng các bộ chuyển đổi của Scikit-Learn để tiền xử lý. Những công cụ này khá tiện lợi, và bạn có thể sẽ sử dụng chúng thường xuyên, đặc biệt là khi khám phá và thử nghiệm với dữ liệu.

Tuy nhiên, khi huấn luyện các mô hình TensorFlow trên các tập dữ liệu lớn, bạn có thể muốn sử dụng API tải và tiền xử lý dữ liệu riêng của TensorFlow, được gọi là `tf.data`. Thật vậy, nó có khả năng tải và tiền xử lý dữ liệu cực kỳ hiệu quả, đọc từ nhiều tệp song song bằng cách sử dụng đa luồng và xếp hàng, xáo trộn và nhóm các mẫu, và nhiều hơn nữa. Thêm vào đó, nó có thể thực hiện tất cả những điều này ngay lập tức – nó tải và tiền xử lý lô dữ liệu tiếp theo trên nhiều lõi CPU, trong khi GPU hoặc TPU của bạn đang huấn luyện lô dữ liệu hiện tại.

API `tf.data` cho phép bạn xử lý các tập dữ liệu không vừa với bộ nhớ và tận dụng tối đa tài nguyên phần cứng, từ đó tăng tốc quá trình huấn luyện. Theo mặc định, API `tf.data` có thể đọc dữ liệu từ các tệp văn bản (như tệp CSV), các tệp nhị phân với các bản ghi có kích thước cố định và các tệp nhị phân sử dụng định dạng TFRecord của TensorFlow, hỗ trợ các bản ghi có kích thước khác nhau.

TFRecord là một định dạng nhị phân linh hoạt và hiệu quả, thường chứa các protocol buffers (một định dạng nhị phân mã nguồn mở). API `tf.data` cũng hỗ trợ đọc dữ liệu từ cơ sở dữ liệu SQL. Hơn nữa, có nhiều tiện ích mở rộng mã nguồn mở để đọc dữ liệu từ nhiều nguồn dữ liệu khác nhau, chẳng hạn như dịch vụ BigQuery của Google (xem <https://tensorflow.org/io>).

Keras cũng đi kèm với các lớp tiền xử lý mạnh mẽ nhưng dễ sử dụng, có thể được nhúng vào mô hình của bạn: bằng cách này, khi bạn triển khai mô hình vào môi trường sản xuất, nó sẽ có thể trực tiếp tiếp nhận dữ liệu thô mà không cần phải thêm bất kỳ mã tiền xử lý bổ sung nào. Điều này loại bỏ nguy cơ không khớp giữa mã tiền xử lý được sử dụng trong quá trình huấn luyện và mã tiền xử lý được sử dụng trong môi trường sản xuất, điều này có thể gây ra sự sai lệch giữa quá trình huấn luyện và phục vụ. Và nếu bạn triển khai mô hình của mình trong nhiều ứng dụng được viết bằng các ngôn ngữ lập trình khác nhau, bạn sẽ không phải viết lại cùng một đoạn mã tiền xử lý nhiều lần, điều này cũng làm giảm nguy cơ không tương thích.

Như chúng ta sẽ thấy, cả hai API có thể được sử dụng kết hợp, ví dụ như để tận dụng khả năng tải dữ liệu hiệu quả do `tf.data` cung cấp và sự tiện lợi của các lớp tiền xử lý Keras.

Trong chương này, trước tiên chúng ta sẽ tìm hiểu về API `tf.data` và định dạng TFRecord. Tiếp theo, chúng ta sẽ tìm hiểu về các lớp tiền xử lý của Keras và cách sử dụng chúng với API `tf.data`. Cuối cùng, chúng ta sẽ xem xét nhanh một vài thư viện liên quan mà bạn có thể thấy hữu ích cho việc tải và tiền xử lý dữ liệu, chẳng hạn như TensorFlow Datasets và TensorFlow Hub. Vậy, hãy bắt đầu nào!

API `tf.data`

Toàn bộ API `tf.data` xoay quanh khái niệm `tf.data.Dataset`: khái niệm này đại diện cho một chuỗi các mục dữ liệu. Thông thường

Bạn sẽ sử dụng các tập dữ liệu đọc dữ liệu dần dần từ đĩa, nhưng để đơn giản, hãy tạo một tập dữ liệu từ một tensor dữ liệu đơn giản bằng cách sử dụng `tf.data.Dataset.from_tensor_slices()`:

```
>>> import tensorflow as tf >>> X =
tf.range(10) # bất kỳ tensor dữ liệu nào >>> dataset =
tf.data.Dataset.from_tensor_slices(X) >>> dataset <TensorSliceDataset shapes: (),
types: tf.int32>
```

Hàm `from_tensor_slices()` nhận một tensor và tạo ra một `tf.data.Dataset` có các phần tử là tất cả các lát cắt của `X` dọc theo chiều thứ nhất, vì vậy tập dữ liệu này chứa 10 mục: các tensor 0, 1, 2, ..., 9. Trong trường hợp này, chúng ta sẽ nhận được cùng một tập dữ liệu nếu chúng ta sử dụng `tf.data.Dataset.range(10)` (ngoại trừ các phần tử sẽ là số nguyên 64 bit thay vì số nguyên 32 bit).

Bạn có thể lặp qua các phần tử của tập dữ liệu như sau:

```
>>> đối với mỗi mục trong tập dữ liệu:
...     in (mục)
...
tf.Tensor(0, shape=(), dtype=int32)
tf.Tensor(1, shape=(), dtype=int32)
[...]
tf.Tensor(9, shape=(), dtype=int32)
```

GHI CHÚ

API `tf.data` là một API xử lý dữ liệu theo luồng: bạn có thể duyệt qua các mục của tập dữ liệu một cách rất hiệu quả, nhưng API này không được thiết kế để lập chỉ mục hoặc cắt lát dữ liệu.

Một tập dữ liệu cũng có thể chứa các bộ giá trị tensor, hoặc các từ điển chứa các cặp tên/tensor, hoặc thậm chí là các bộ giá trị và từ điển tensor lồng nhau. Khi cắt một bộ giá trị, một từ điển hoặc một cấu trúc lồng nhau, tập dữ liệu sẽ chỉ cắt các tensor mà nó chứa, đồng thời bảo toàn cấu trúc bộ giá trị/từ điển. Ví dụ:

```
>>> X_nested = {"a": ([1, 2, 3], [4, 5, 6]), "b": [7, 8, 9]} >>> dataset
= tf.data.Dataset.from_tensor_slices(X_nested) >>> for item in dataset:

...     in (mục)
...
{'a': (<tf.Tensor: [...]=1>, <tf.Tensor: [...]=4>), 'b': <tf.Tensor:
[...]=7>} {'a':
(<tf.Tensor: [...]=2>, <tf.Tensor: [...]=5>), 'b': <tf.Tensor:
[...]=8>} {'a':
(<tf.Tensor: [...]=3>, <tf.Tensor: [...]=6>), 'b': <tf.Tensor:
[...]=9>}
```

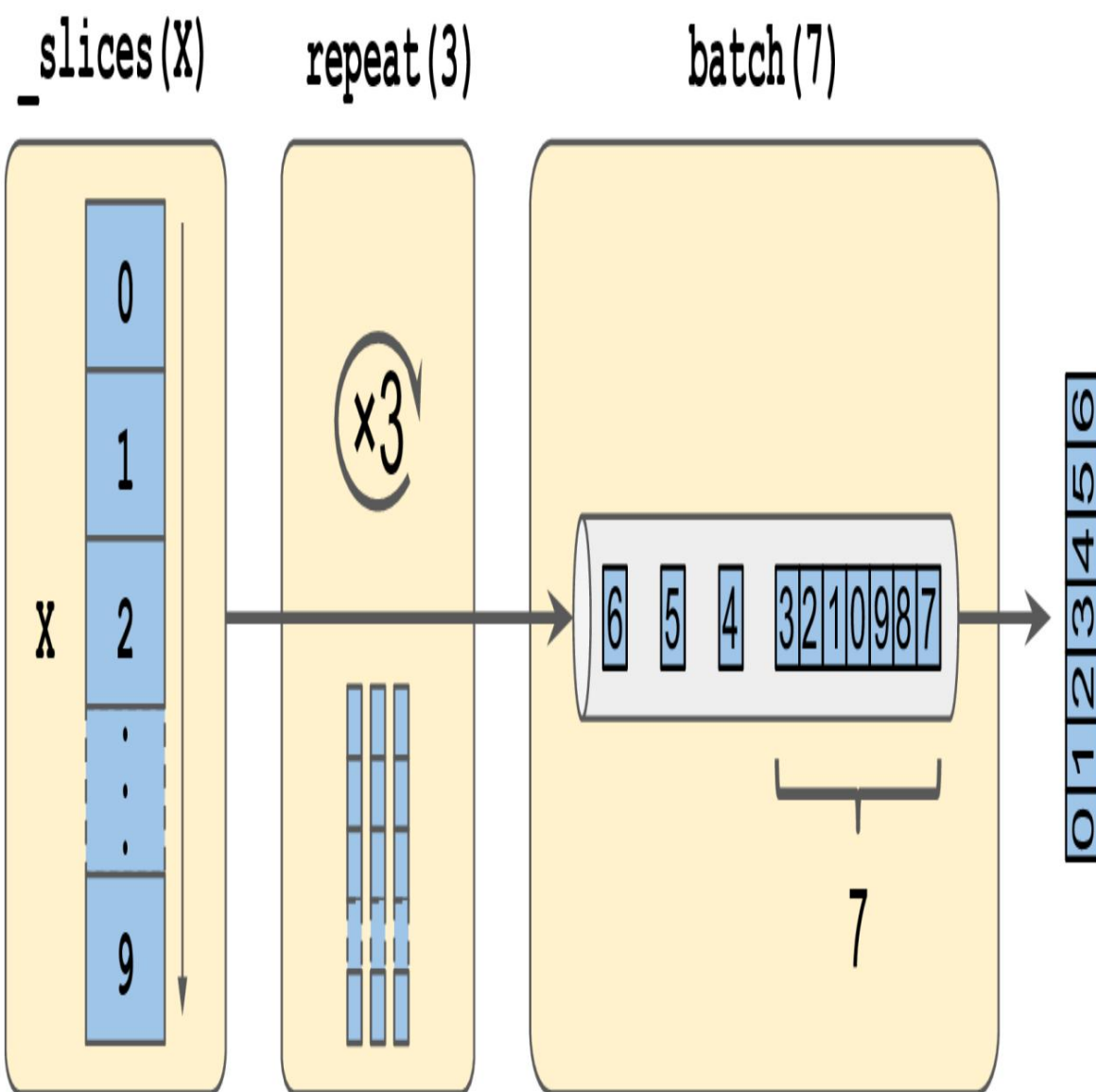
Chuỗi biến đổi

Khi đã có tập dữ liệu, bạn có thể áp dụng đủ loại phép biến đổi cho nó bằng cách gọi các phương thức biến đổi của tập dữ liệu đó. Mỗi phương thức trả về một tập dữ liệu mới, vì vậy bạn có thể xích chuỗi các phép biến đổi như thế này (chuỗi này được minh họa trong [Hình 13-1](#)):

```
>>> dataset = tf.data.Dataset.from_tensor_slices(tf.range(10)) >>> dataset =
dataset.repeat(3).batch(7) >>> for item in dataset:

...     in (mục)
...
tf.Tensor([0 1 2 3 4 5 6], shape=(7,), dtype=int32)
tf.Tensor([7 8 9 0 1 2 3], shape=(7,), dtype=int32)
tf.Tensor([4 5 6 7 8 9 0], shape=(7,), dtype=int32)
tf.Tensor([1 2 3 4 5 6 7], shape=(7,), dtype=int32)
tf.Tensor([8 9], shape=(2,), dtype=int32)
```

from_tensor



Hình 13-1. Chuỗi các phép biến đổi tập dữ liệu

Trong ví dụ này, trước tiên chúng ta gọi phương thức `repeat()` trên tập dữ liệu gốc, và nó trả về một tập dữ liệu mới sẽ lặp lại các mục của tập dữ liệu gốc ba lần. Tất nhiên, điều này sẽ không sao chép tất cả dữ liệu trong bộ nhớ ba lần! Nếu bạn gọi phương thức này mà không có đối số, tập dữ liệu mới sẽ lặp lại tập dữ liệu nguồn mãi mãi, vì vậy đoạn mã lặp qua tập dữ liệu sẽ phải quyết định khi nào dừng lại.

Tiếp theo, chúng ta gọi phương thức `batch()` trên tập dữ liệu mới này, và thao tác này lại tạo ra một tập dữ liệu mới. Tập dữ liệu này sẽ nhóm các mục của tập dữ liệu trước đó thành từng nhóm bảy mục.

Cuối cùng, chúng ta lặp lại các mục của tập dữ liệu cuối cùng này. Phương thức `batch()` phải xuất ra một lô cuối cùng có kích thước là hai thay vì bảy, nhưng bạn có thể gọi `batch()` với `drop_remainder=True` nếu muốn nó loại bỏ lô cuối cùng này, sao cho tất cả các lô có cùng kích thước.

CẢNH BÁO

Các phương thức của tập dữ liệu không sửa đổi tập dữ liệu hiện có mà tạo ra các tập dữ liệu mới. Vì vậy, hãy đảm bảo giữ lại tham chiếu đến các tập dữ liệu mới này (ví dụ: với `dataset = ...`), nếu không sẽ không có gì xảy ra.

Bạn cũng có thể biến đổi các mục bằng cách gọi phương thức `map()`. Ví dụ, điều này tạo ra một tập dữ liệu mới với tất cả các lô được nhân đôi:

```
>>> dataset = dataset.map(lambda x: x * 2) # x là một lô
item in dataset:
...     in (mục)
...
tf.Tensor([ 0 2 4 6 8 10 12], shape=(7,), dtype=int32)
tf.Tensor([14 16 18 0 2 4 6], shape=(7,), dtype=int32) [...]
```

Phương thức `map()` này là phương thức bạn sẽ gọi để áp dụng bất kỳ quá trình tiền xử lý nào cho dữ liệu của mình. Đôi khi điều này sẽ bao gồm các phép tính khá tốn tài nguyên, chẳng hạn như định hình lại hoặc xoay ảnh, vì vậy bạn thường muốn tạo nhiều luồng để tăng tốc độ. Điều này có thể được thực hiện bằng cách đặt đối số `num_parallel_calls` thành số luồng cần chạy, hoặc bằng `tf.data.AUTOTUNE`. Lưu ý rằng hàm bạn truyền cho phương thức `map()` phải có thể chuyển đổi thành một hàm TF (xem [Chương 12](#)).

Cũng có thể lọc tập dữ liệu một cách đơn giản bằng phương thức `filter()`.

Ví dụ, đoạn mã này tạo ra một tập dữ liệu chỉ chứa các lô có tổng lớn hơn 50:

```
>>> dataset = dataset.filter(lambda x: tf.reduce_sum(x) > 50) >>> for item
in dataset:
...     in (mục)
...
tf.Tensor([14 16 18 0 2 4 6], shape=(7,), dtype=int32) tf.Tensor([ 8
10 12 14 16 18 0], shape=(7,), dtype=int32) tf.Tensor([ 2 4 6 8 10 12
14], shape=(7,), dtype=int32)
```

Bạn thường muốn chỉ xem một vài mục từ một tập dữ liệu. Bạn có thể sử dụng phương thức `take()` cho việc đó:

```
>>> for item in dataset.take(2):
...     print(item)
...
tf.Tensor([14 16 18 0 2 4 6], shape=(7,), dtype=int32) tf.Tensor([ 8
10 12 14 16 18 0], shape=(7,), dtype=int32)
```

Xáo trộn dữ liệu

Như chúng ta đã thảo luận trong [Chương 4](#), thuật toán Gradient Descent hoạt động tốt nhất khi các trường hợp trong tập huấn luyện là độc lập và phân bố đồng nhất (IID). Một cách đơn giản để đảm bảo điều này là xáo trộn các trường hợp bằng cách sử dụng phương thức `shuffle()`. Phương thức này sẽ tạo ra một tập dữ liệu mới, bắt đầu bằng cách điền vào bộ đệm các mục đầu tiên của tập dữ liệu nguồn. Sau đó, bất cứ khi nào được yêu cầu một mục, nó sẽ lấy ngẫu nhiên một mục từ bộ đệm và thay thế bằng một mục mới từ tập dữ liệu nguồn, cho đến khi nó lặp qua toàn bộ tập dữ liệu nguồn. Tại thời điểm này, nó sẽ tiếp tục lấy ngẫu nhiên các mục từ bộ đệm cho đến khi bộ đệm trống. Bạn phải chỉ định kích thước bộ đệm, và điều quan trọng là phải làm cho nó đủ lớn, nếu không việc xáo trộn sẽ không hiệu quả¹. Chỉ cần không vượt quá dung lượng RAM bạn có, và ngay cả khi bạn có nhiều RAM, cũng không cần thiết phải vượt quá kích thước của tập dữ liệu. Bạn có thể cung cấp một hạt giống ngẫu nhiên nếu muốn thứ tự ngẫu nhiên giống nhau mỗi khi chạy chương trình. Ví dụ, đoạn mã sau tạo và hiển thị một tập dữ liệu chứa các số nguyên từ 0 đến 9, được lặp lại hai lần, được xáo trộn bằng bộ đệm có kích thước 4 và hạt giống ngẫu nhiên là 42, và được nhóm lại với kích thước lô là 7:

```
>>> dataset = tf.data.Dataset.range(10).repeat(2) >>> dataset =
dataset.shuffle(buffer_size=4, seed=42).batch(7) >>> for item in dataset:
```

```

...         in (mục)
...
tf.Tensor([3 0 1 6 2 5 7], shape=(7,), dtype=int64)
tf.Tensor([8 4 1 9 4 2 3], shape=(7,), dtype=int64)
tf.Tensor([7 5 0 8 9 6], shape=(6,), dtype=int64)

```

MẸO

Nếu bạn gọi hàm `repeat()` trên một tập dữ liệu đã được xáo trộn, theo mặc định nó sẽ tạo ra một thứ tự mới ở mỗi lần lặp. Điều này nhìn chung là tốt, nhưng nếu bạn muốn sử dụng lại cùng một thứ tự ở mỗi lần lặp (ví dụ: để kiểm tra hoặc gỡ lỗi), bạn có thể đặt `reshuffle_each_iteration=False` khi gọi hàm `shuffle()`.

Đối với tập dữ liệu lớn không vừa bộ nhớ, phương pháp xáo trộn bộ đệm đơn giản này có thể không đủ, vì bộ đệm sẽ nhỏ so với tập dữ liệu. Một giải pháp là xáo trộn chính dữ liệu nguồn (ví dụ, trên Linux bạn có thể xáo trộn các tệp văn bản bằng lệnh `shuf`). Điều này chắc chắn sẽ cải thiện việc xáo trộn rất nhiều! Ngay cả khi dữ liệu nguồn đã được xáo trộn, bạn thường vẫn muốn xáo trộn thêm nữa, nếu không thứ tự sẽ lặp lại ở mỗi epoch, và mô hình có thể bị sai lệch (ví dụ, do một số mẫu không mong muốn xuất hiện ngẫu nhiên trong thứ tự dữ liệu nguồn). Để xáo trộn các trường hợp thêm nữa, một phương pháp phổ biến là chia dữ liệu nguồn thành nhiều tệp, sau đó đọc chúng theo thứ tự ngẫu nhiên trong quá trình huấn luyện.

Tuy nhiên, các bản ghi nằm trong cùng một tệp vẫn sẽ nằm gần nhau. Để tránh điều này, bạn có thể chọn ngẫu nhiên nhiều tệp và đọc chúng đồng thời, xen kẽ các bản ghi. Sau đó, bạn có thể thêm bộ đệm xáo trộn bằng phương thức ``shuffle()``. Nếu tất cả những điều này nghe có vẻ phức tạp, đừng lo lắng: API ``tf.data`` cho phép thực hiện tất cả điều này chỉ với vài dòng mã. Hãy cùng xem cách bạn có thể làm điều này.

Xen kẽ các dòng từ nhiều tập tin Đầu tiên, giả

sử bạn đã tải bộ dữ liệu nhà ở California, xáo trộn nó (trừ khi nó đã được xáo trộn sẵn), và chia nó thành một tập huấn luyện, một tập xác thực và một tập kiểm tra. Sau đó, bạn chia mỗi tập thành nhiều tệp CSV mà mỗi tệp con chứa dữ liệu của bạn.

trông như thế này (mỗi hàng chứa tám đặc điểm đầu vào cộng với giá trị nhà trung bình mục tiêu):

```
MedInc,HouseAge,AveRooms,AveBedrms,Popul.,AveOccup,Lat.,Long.,MedianHouseValue
```

```
3.5214,15.0,3.050,1.107,1447.0,1.606,37.63,-122.43,1.442
```

```
5.3275,5.0,6.490,0.991,3464.0,3.443,33.69,-117.39,1.687
```

```
3.1,29.0,7.542,1.592,1328.0,2.251,38.44,-122.98,1.621 [...]
```

Giả sử `train_filepaths` chứa danh sách các đường dẫn tệp huấn luyện (và bạn cũng có `valid_filepaths` và `test_filepaths`):

```
>>> train_filepaths
['datasets/housing/my_train_00.csv', 'datasets/
housing/my_train_01.csv', ...]
```

Ngoài ra, bạn cũng có thể sử dụng các mẫu đường dẫn tệp; ví

dụ: `train_filepaths =`

`"datasets/housing/my_train_*.csv"`. Bây giờ, hãy tạo một tập dữ liệu chỉ chứa các đường dẫn tệp này:

```
filepath_dataset = tf.data.Dataset.list_files(train_filepaths, seed=42)
```

Theo mặc định, hàm `list_files()` trả về một tập dữ liệu đã xáo trộn các đường dẫn tệp. Nói chung, đây là điều tốt, nhưng bạn có thể đặt `shuffle=False` nếu vì lý do nào đó bạn không muốn điều đó xảy ra.

Tiếp theo, bạn có thể gọi phương thức `interleave()` để đọc dữ liệu từ năm tệp cùng một lúc và xen kẽ các dòng của chúng. Bạn cũng có thể bỏ qua dòng đầu tiên của mỗi tệp – là dòng tiêu đề – bằng cách sử dụng phương thức `skip()`.

```
n_readers = 5
dataset = filepath_dataset.interleave( lambda
    filepath: tf.data.TextLineDataset(filepath).skip(1), cycle_length=n_readers)
```

Phương thức `interleave()` sẽ tạo một tập dữ liệu lấy năm đường dẫn tệp từ `filepath_dataset`, và với mỗi đường dẫn, nó sẽ gọi hàm bạn cung cấp (một hàm lambda trong ví dụ này) để tạo một tập dữ liệu mới (trong trường hợp này là `TextLineDataset`). Nói rõ hơn, ở giai đoạn này sẽ có tổng cộng bảy tập dữ liệu: tập dữ liệu `filepath`, tập dữ liệu `interleave`, và năm `TextLineDataset` được tạo nội bộ bởi tập dữ liệu `interleave`. Khi chúng ta lặp qua tập dữ liệu `interleave`, nó sẽ duyệt qua năm `TextLineDataset` này, đọc từng dòng một từ mỗi tập dữ liệu cho đến khi tất cả các tập dữ liệu hết phần tử. Sau đó, nó sẽ lấy năm đường dẫn tệp tiếp theo từ `filepath_dataset` và xen kẽ chúng theo cùng một cách, và cứ thế tiếp tục cho đến khi hết đường dẫn tệp. Để việc xen kẽ hoạt động tốt nhất, nên sử dụng các tệp có độ dài giống nhau; nếu không, phần cuối của tệp dài nhất sẽ không được xen kẽ.

Theo mặc định, hàm `interleave()` không sử dụng song song; nó chỉ đọc từng dòng một từ mỗi tệp, theo trình tự. Nếu bạn muốn nó thực sự đọc các tệp song song, bạn có thể đặt đối số `num_parallel_calls` của phương thức `interleave()` thành số luồng bạn muốn (hãy nhớ rằng phương thức `map()` cũng có đối số này). Bạn thậm chí có thể đặt nó thành `tf.data.AUTOTUNE` để TensorFlow tự động chọn số luồng phù hợp dựa trên CPU khả dụng. Bây giờ chúng ta hãy xem tập dữ liệu chứa gì:

```
>>> for line in dataset.take(5):
...     print(line)
...
tf.Tensor(b'4.5909,16.0,[...],33.63,-117.71,2.418', shape=(),
dtype=string)
tf.Tensor(b'2.4792,24.0,[...],34.18,-118.38,2.0', shape=(),
dtype=string)
tf.Tensor(b'4.2708,45.0,[...],37.48,-122.19,2.67', shape=(),
dtype=string)
tf.Tensor(b'2.1856,41.0,[...],32.76,-117.12,1.205', shape=(),
dtype=string)
tf.Tensor(b'4.1812,52.0,[...],33.73,-118.31,3.215', shape=(),
dtype=string)
```

Đây là những hàng đầu tiên (không tính hàng tiêu đề) của năm tệp CSV được chọn ngẫu nhiên.

Trông ổn đấy!

GHI CHÚ

Bạn có thể truyền một danh sách các đường dẫn tệp vào hàm tạo của `TextLineDataset`: nó sẽ duyệt qua từng tệp theo thứ tự, từng dòng một. Nếu bạn cũng đặt đối số `num_parallel_reads` thành một số lớn hơn một, thì tập dữ liệu sẽ đọc song song số lượng tệp đó và xen kẽ các dòng của chúng (mà không cần phải gọi phương thức `interleave()`). Tuy nhiên, nó sẽ không xáo trộn các tệp tin, cũng như không bỏ qua các dòng tiêu đề.

Xử lý sơ bộ dữ liệu

Giờ đây, chúng ta đã có một tập dữ liệu về nhà ở trả về mỗi trường hợp dưới dạng một tensor chứa chuỗi byte, chúng ta cần thực hiện một số bước tiền xử lý, bao gồm phân tích chuỗi và chuẩn hóa dữ liệu. Hãy cùng triển khai một vài hàm tùy chỉnh để thực hiện quá trình tiền xử lý này:

```
X_mean, X_std = [...] # giá trị trung bình và hệ số tỷ lệ của mỗi đặc trưng trong tập huấn luyện
n_inputs = 8

def parse_csv_line(line):
    defs = [0.] * n_inputs + [tf.constant([], dtype=tf.float32)]
    fields = tf.io.decode_csv(line, record_defaults=defs)
    return tf.stack(fields[:-1]), tf.stack(fields[-1:])

def preprocess(line):
    x, y = parse_csv_line(line)
    return (x - X_mean) / X_std, y
```

Hãy cùng phân tích đoạn mã này:

- Đầu tiên, đoạn mã giả định rằng chúng ta đã tính toán trước giá trị trung bình và độ lệch chuẩn của mỗi đặc trưng trong tập huấn luyện. `X_mean` và `X_std` chỉ là các tensor 1 chiều (hoặc mảng NumPy) chứa tám số thực, mỗi số thực tương ứng với một đặc trưng đầu vào. Điều này có thể được thực hiện bằng cách sử dụng Scikit-Learn.

StandardScaler trên một mẫu ngẫu nhiên đủ lớn của tập dữ liệu.

Ở phần sau của chương này, chúng ta sẽ sử dụng lớp tiền xử lý Keras.

- Hàm `parse_csv_line()` nhận vào một dòng CSV và phân tích nó. Để hỗ trợ việc đó, nó sử dụng hàm `tf.io.decode_csv()`, hàm này nhận hai đối số: đối số đầu tiên là dòng cần phân tích cú pháp, và đối số thứ hai là một mảng chứa giá trị mặc định cho mỗi cột trong tệp CSV. Mảng này (defs) cho TensorFlow biết không chỉ giá trị mặc định cho mỗi cột, mà còn cả số lượng cột và kiểu dữ liệu của chúng. Trong ví dụ này, chúng ta cho biết tất cả các cột đặc trưng đều là số thực và các giá trị thiếu nên mặc định là 0, nhưng chúng ta cung cấp một mảng rỗng thuộc kiểu `tf.float32` làm giá trị mặc định cho cột cuối cùng (mục tiêu): mảng này cho TensorFlow biết rằng cột này chứa các số thực, nhưng không có giá trị mặc định, vì vậy nó sẽ báo lỗi nếu gặp phải giá trị thiếu.
- Hàm `tf.io.decode_csv()` trả về một danh sách các tensor vô hướng (một tensor cho mỗi cột), nhưng chúng ta cần trả về một mảng tensor 1 chiều. Vì vậy, chúng ta gọi `tf.stack()` trên tất cả các tensor ngoại trừ tensor cuối cùng (giá trị mục tiêu): thao tác này sẽ xếp chồng các tensor này thành một mảng 1 chiều. Sau đó, chúng ta làm tương tự với giá trị mục tiêu: thao tác này biến nó thành một mảng tensor 1 chiều với một giá trị duy nhất, thay vì một tensor vô hướng. Hàm `tf.io.decode_csv()` đã hoàn tất, vì vậy nó trả về các đặc trưng đầu vào và giá trị mục tiêu.
- Cuối cùng, hàm `preprocess()` tùy chỉnh chỉ gọi hàm `parse_csv_line()`, chuẩn hóa các đặc trưng đầu vào bằng cách trừ đi giá trị trung bình của các đặc trưng rồi chia cho độ lệch chuẩn của các đặc trưng, và trả về một bộ giá trị chứa các đặc trưng đã được chuẩn hóa và giá trị mục tiêu.

Hãy thử nghiệm hàm tiền xử lý này:

```
>>>
```

```
preprocess(b'4.2083,44.0,5.3232,0.9171,846.0,2.3370,37.47,-122.2, 2.782')
```

```
(<tf.Tensor: shape=(8,), dtype=float32, numpy=
array([ 0.16579159, 1.216324 , -0.05204564, -0.39215982,
```

```
-0.5277444 ,
      -0.2633488 , 0.8543046 , -1.3072058 ], dtype=float32)>,
<tf.Tensor: shape=(1,), dtype=float32, numpy=array([2.782], dtype=float32)>)
```

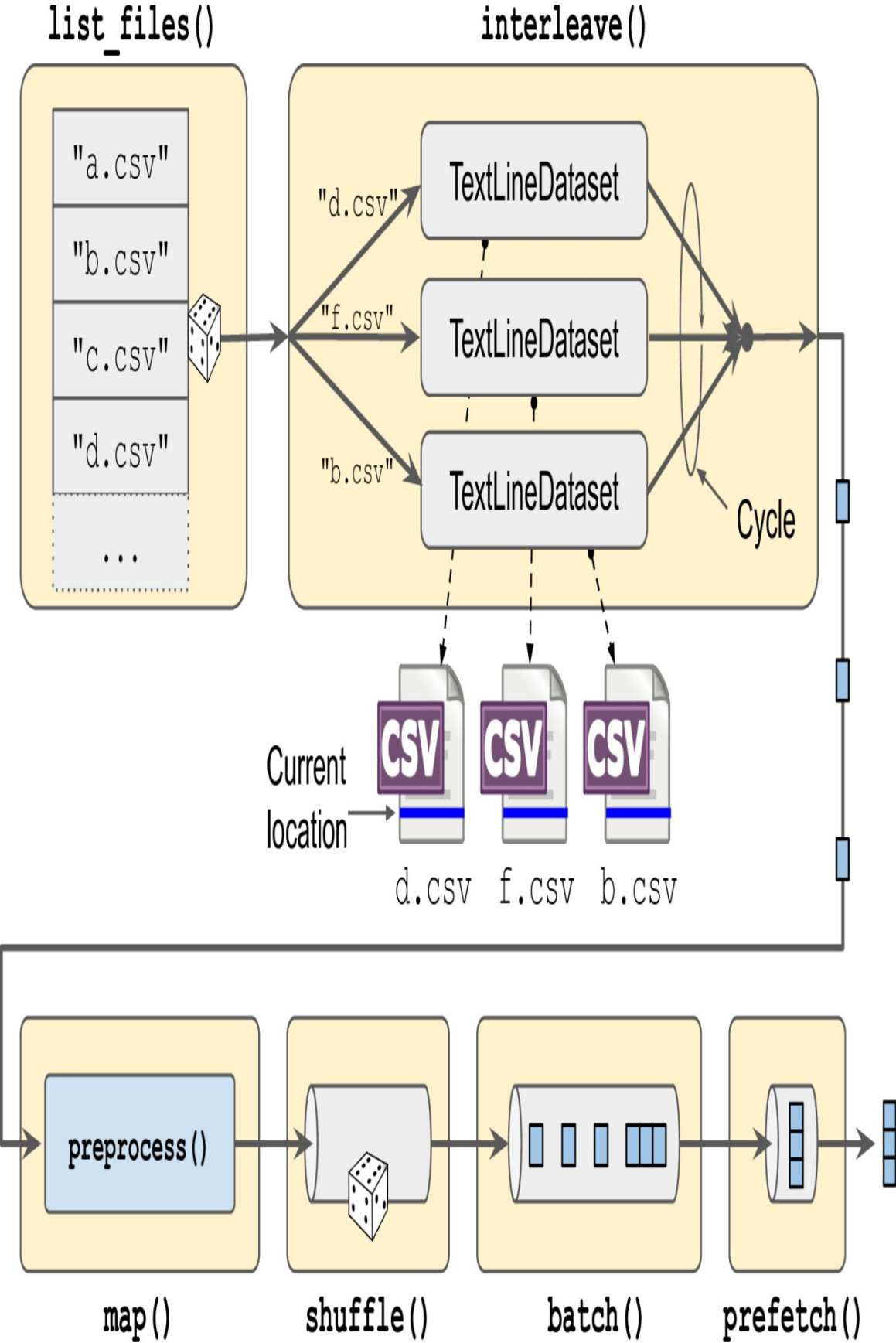
Trông tuyệt vời! Hàm `preprocess()` có thể chuyển đổi một đối tượng từ chuỗi byte thành một tensor được hiệu chỉnh tốt, với nhãn tương ứng. Giờ đây, chúng ta có thể sử dụng phương thức `map()` của tập dữ liệu để áp dụng hàm `preprocess()` cho từng mẫu trong tập dữ liệu.

Ghép mọi thứ lại với nhau

Để làm cho mã nguồn dễ tái sử dụng hơn, chúng ta hãy kết hợp tất cả những gì đã thảo luận cho đến nay vào một hàm hỗ trợ khác: hàm này sẽ tạo và trả về một tập dữ liệu có khả năng tải dữ liệu nhà ở California một cách hiệu quả từ nhiều tệp CSV, tiền xử lý, xáo trộn và xử lý theo lô (xem [Hình 13-2](#)):

```
def csv_reader_dataset(filepaths, n_readers=5,
                       n_read_threads=None,
                           n_parse_threads=5,
shuffle_buffer_size=10_000, seed=42,
                           batch_size=32):
    dataset = tf.data.Dataset.list_files(filepaths, seed=seed)
    dataset = dataset.interleave( lambda
        filepath:
    tf.data.TextLineDataset(filepath).skip(1),
        cycle_length=n_readers,
    num_parallel_calls=n_read_threads)
    tập dữ liệu = tập dữ liệu.map(tiền xử lý,
    num_parallel_calls=n_parse_threads)
    dataset = dataset.shuffle(shuffle_buffer_size, seed=seed)
    return dataset.batch(batch_size).prefetch(1)
```

Lưu ý rằng chúng ta sử dụng phương thức `prefetch()` ở dòng cuối cùng. Điều này rất quan trọng đối với hiệu suất, như chúng ta sẽ thấy ngay sau đây.



Hình 13-2. Tải và tiền xử lý dữ liệu từ nhiều tệp CSV.

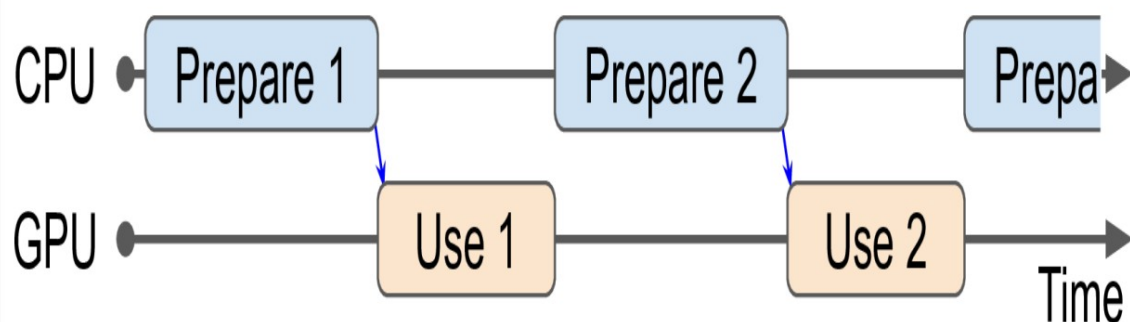
Tải trước Bằng

cách gọi `prefetch(1)` ở cuối hàm `csv_reader_dataset()`

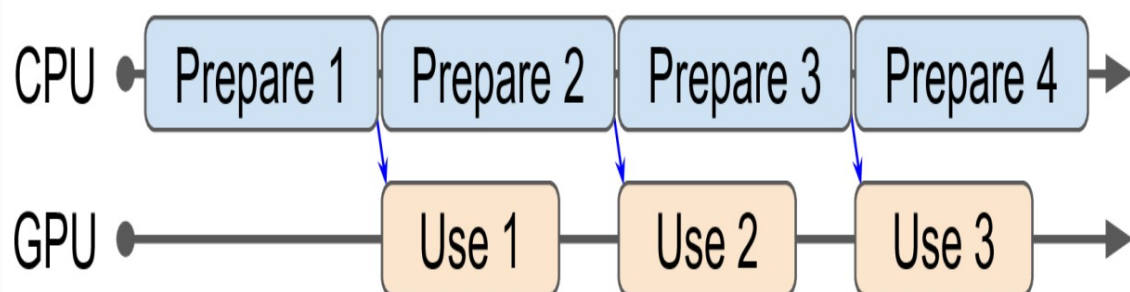
tùy chỉnh, chúng ta đang tạo một tập dữ liệu sẽ cố gắng hết sức để luôn đi trước một lô. Nói cách khác, trong khi thuật toán huấn luyện của chúng ta đang xử lý một lô, tập dữ liệu sẽ hoạt động song song để chuẩn bị lô tiếp theo (ví dụ: đọc dữ liệu từ đĩa và xử lý trước). Điều này có thể cải thiện hiệu suất đáng kể, như được minh họa trong Hình 13-3.

Nếu chúng ta cũng đảm bảo rằng việc tải và tiền xử lý được thực hiện đa luồng (bằng cách thiết lập `num_parallel_calls` khi gọi `interleave()` và `map()`), chúng ta có thể tận dụng nhiều lõi CPU và hy vọng sẽ làm cho việc chuẩn bị một lô dữ liệu ngắn hơn so với việc chạy một bước huấn luyện trên GPU: theo cách này, GPU sẽ được sử dụng gần như 100% (ngoại trừ thời gian truyền dữ liệu từ CPU sang GPU), và quá trình huấn luyện sẽ chạy nhanh hơn nhiều.

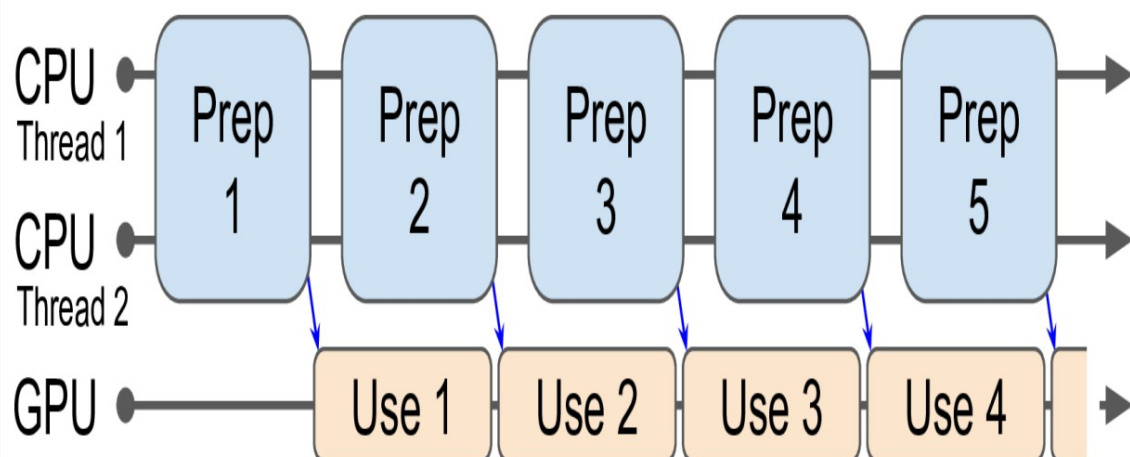
Without prefetching



With prefetching



With prefetching + multithreaded loading & preprocessing



Hình 13-3. Với tính năng tìm nạp trước, CPU và GPU hoạt động song song: khi GPU xử lý một lô dữ liệu, CPU sẽ xử lý lô tiếp theo.

MẸO

Nếu bạn dự định mua card đồ họa (GPU), thì sức mạnh xử lý và dung lượng bộ nhớ của nó dĩ nhiên là rất quan trọng (đặc biệt, dung lượng RAM lớn là rất cần thiết cho các mô hình xử lý hình ảnh máy tính hoặc xử lý ngôn ngữ tự nhiên quy mô lớn). Quan trọng không kém đối với hiệu năng tốt là băng thông bộ nhớ của GPU; đây là số gigabyte dữ liệu mà nó có thể truyền vào hoặc ra khỏi RAM mỗi giây.

Nếu tập dữ liệu đủ nhỏ để vừa với bộ nhớ, bạn có thể tăng tốc đáng kể quá trình huấn luyện bằng cách sử dụng phương thức `cache()` của tập dữ liệu để lưu trữ nội dung của nó vào RAM. Thông thường, bạn nên thực hiện việc này sau khi tải và tiền xử lý dữ liệu, nhưng trước khi xáo trộn, lặp lại, chia thành các lô và tải trước dữ liệu. Bằng cách này, mỗi trường hợp sẽ chỉ được đọc và tiền xử lý một lần (thay vì một lần mỗi epoch), nhưng dữ liệu vẫn sẽ được xáo trộn khác nhau ở mỗi epoch, và lô tiếp theo vẫn sẽ được chuẩn bị trước.

Giờ đây bạn đã học cách xây dựng các pipeline đầu vào hiệu quả để tải và tiền xử lý dữ liệu từ nhiều tệp văn bản. Chúng ta đã thảo luận về các phương thức tập dữ liệu phổ biến nhất, nhưng vẫn còn một vài phương thức khác mà bạn có thể muốn tìm hiểu, chẳng hạn như `concatenate()`, `zip()`, `window()`, `reduce()`, `shard()`, `flat_map()`, `apply()`, `unbatch()` và `padded_batch()`. Ngoài ra còn có một vài phương thức lớp khác, chẳng hạn như `from_generator()` và `from_tensors()`, lần lượt tạo ra một tập dữ liệu mới từ một generator Python hoặc một danh sách các tensor. Vui lòng kiểm tra tài liệu API để biết thêm chi tiết. Cũng cần lưu ý rằng có các tính năng thử nghiệm có sẵn trong `tf.data.experimental`, nhiều tính năng trong số đó có thể sẽ được đưa vào API cốt lõi trong các bản phát hành tương lai (ví dụ: hãy xem lớp `CsvDataset`, cũng như phương thức `make_csv_dataset()`, phương thức này sẽ xử lý việc suy luận kiểu dữ liệu của từng cột).

Sử dụng Dataset với Keras Giờ đây,

chúng ta có thể sử dụng hàm `csv_reader_dataset()` tùy chỉnh mà bạn đã viết trước đó để tạo một dataset cho tập huấn luyện, tập xác thực và tập kiểm tra. Tập huấn luyện sẽ được xáo trộn ở mỗi epoch (lưu ý rằng tập xác thực và tập kiểm tra cũng sẽ được xáo trộn, mặc dù chúng ta không thực sự cần điều đó).

```
train_set = csv_reader_dataset(train_filepaths) valid_set
= csv_reader_dataset(valid_filepaths) test_set =
csv_reader_dataset(test_filepaths)
```

Giờ đây, bạn có thể dễ dàng xây dựng và huấn luyện mô hình Keras bằng cách sử dụng các tập dữ liệu này. Khi gọi phương thức `fit()` của mô hình, bạn có thể truyền `train_set` thay vì `X_train, y_train`, và truyền `valid_data=valid_set` thay vì `valid_data=(X_valid, y_valid)`. Phương thức `fit()` sẽ tự động lặp lại quá trình huấn luyện trên tập dữ liệu một lần mỗi epoch, sử dụng thứ tự ngẫu nhiên khác nhau ở mỗi epoch.

```
model = tf.keras.Sequential([...])
model.compile(loss="mse", optimizer="sgd")
model.fit(train_set, validation_data=valid_set, epochs=5)
```

Tương tự, bạn có thể truyền một tập dữ liệu cho các phương thức `evaluate()` và `predict()`:

```
test_mse = model.evaluate(test_set) new_set = test_set.take(3)
# giả vờ chúng ta có 3 mẫu mới y_pred = model.predict(new_set) # hoặc bạn có thể chỉ cần truyền một
mảng NumPy
```

Khác với các tập dữ liệu khác, tập dữ liệu mới (`new_set`) thường sẽ không chứa nhãn. Nếu có, như trường hợp này, Keras sẽ bỏ qua chúng. Lưu ý rằng trong tất cả các trường hợp này, bạn vẫn có thể sử dụng mảng NumPy thay vì tập dữ liệu nếu muốn (nhưng tất nhiên chúng cần được tải và xử lý trước).

Nếu bạn muốn xây dựng vòng lặp huấn luyện tùy chỉnh của riêng mình (như trong [Chương 12](#)), bạn chỉ cần lặp lại trên tập dữ liệu huấn luyện, một cách rất tự nhiên:

```
n_epochs = 5
for epoch in range(n_epochs):
    for X_batch, y_batch in train_set: [...] #
        thực hiện một bước Gradient Descent
```

Trên thực tế, thậm chí có thể tạo ra một hàm TF (xem [Chương 12](#)) để huấn luyện mô hình trong toàn bộ một epoch. Điều này thực sự có thể tăng tốc quá trình huấn luyện:

```
@tf.function
def train_one_epoch(model, optimizer, loss_fn, train_set):
    for X_batch, y_batch in train_set:
        with tf.GradientTape() as tape:
            y_pred = model(X_batch)
            main_loss = loss_fn(y_batch, y_pred)
            loss = tf.reduce_mean(main_loss)
            tf.add_n([main_loss] + model.losses)
            gradients = tape.gradient(loss, model.trainable_variables)
            optimizer.apply_gradients(zip(gradients, model.trainable_variables))

optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
loss_fn = tf.keras.losses.mean_squared_error
for epoch in range(n_epochs):
    print("\rEpoch {}/{}".format(epoch + 1, n_epochs), end="")
    train_one_epoch(model, optimizer, loss_fn, train_set)
```

Trong Keras, tham số `steps_per_execution` của phương thức `compile()` cho phép bạn xác định số lượng batch mà phương thức `fit()` sẽ xử lý trong mỗi lần gọi đến `tf.function` mà nó sử dụng để huấn luyện. Giá trị mặc định là 1, vì vậy nếu bạn đặt nó thành 50, bạn thường sẽ thấy hiệu suất được cải thiện đáng kể. Tuy nhiên, các phương thức `on_batch_*` của các hàm callback trong Keras sẽ chỉ được gọi sau mỗi 50 batch.

Chúc mừng, giờ bạn đã biết cách xây dựng các đường dẫn dữ liệu đầu vào mạnh mẽ bằng API `tf.data`! Tuy nhiên, cho đến nay chúng ta đã sử dụng các tệp CSV, vốn phổ biến, đơn giản và tiện lợi nhưng thực sự hiệu quả và không hỗ trợ...

TFRecords xử lý tốt các cấu trúc dữ liệu lớn hoặc phức tạp (như hình ảnh hoặc âm thanh). Vì vậy, hãy cùng xem cách sử dụng TFRecords thay thế.

MẸO

Nếu bạn hài lòng với các tệp CSV (hoặc bất kỳ định dạng nào khác bạn đang sử dụng), bạn không cần phải dùng TFRecords. Như người ta vẫn nói, nếu nó không hỏng thì đừng sửa! TFRecords hữu ích khi điểm nghẽn trong quá trình huấn luyện là việc tải và phân tích dữ liệu.

Định dạng TFRecord

Định dạng TFRecord là định dạng được TensorFlow ưu tiên sử dụng để lưu trữ lượng lớn dữ liệu và đọc chúng một cách hiệu quả. Đây là một định dạng nhị phân rất đơn giản, chỉ chứa một chuỗi các bản ghi nhị phân có kích thước khác nhau (mỗi bản ghi bao gồm độ dài, mã kiểm tra CRC để đảm bảo độ dài không bị lỗi, sau đó là dữ liệu thực tế và cuối cùng là mã kiểm tra CRC cho dữ liệu).

Bạn có thể dễ dàng tạo tệp TFRecord bằng cách sử dụng lớp `tf.io.TFRecordWriter`:

```
with tf.io.TFRecordWriter("my_data.tfrecord") as f: f.write(b"Đây là  
bản ghi đầu tiên") f.write(b"Và đây là bản ghi  
thứ hai")
```

Sau đó, bạn có thể sử dụng `tf.data.TFRecordDataset` để đọc một hoặc nhiều tệp TFRecord:

```
filepaths = ["my_data.tfrecord"] dataset =  
tf.data.TFRecordDataset(filepaths) for item in dataset:  
print(item)
```

Kết quả đầu ra sẽ là:

```
tf.Tensor(b'Đây là bản ghi đầu tiên', shape=(), dtype=string) tf.Tensor(b'Và đây  
là bản ghi thứ hai', shape=(), dtype=string)
```